Databases I

Semester 3 (Theory)

Mathis Neunzig

DHBW Mosbach

Autumn 2026

Table of Contents

Table of Contents

About the Lecturer

Mathis Neunzig

- B.Sc. Business Information Technology (DHBW Mannheim)
- M.Sc. Data Science (Fernuniversität Hagen)
- ABAP Developer & Trainer at SAP
- Teaches Databases & Software Engineering at DHBW
- Based in Mannheim (Baden-Württemberg)
- Originally from Oldenburg (Lower Saxony)

Contact

Email
mathis.neunzig@gmail.com

Phone
+49 174 9885992

Getting to Know You

Let's go around the room – please tell us:

1. Your **name**
2. Your **employer**
3. Your **favourite animal**
4. Your **favourite song, album or artist** right now
5. Your **favourite programming language** (so far)

Agenda

This course is structured around multiple interconnected topic groups that build on each other progressively throughout the semester.

1. Foundations

- Databases and Database Management Systems (DBMS)
- Data models
- Relational algebra
- Relational databases
- Data modelling & the DB design process

2. SQL

- Data Manipulation Language (DML)
- Data Definition Language (DDL)
- Data Control Language (DCL)

3. Advanced Topics

- Database transaction management
- Concurrency control
- Performance optimisation
- Backup & Recovery
- Security & SQL Injection
- Object-Relational Mapping (ORM)

4. DB Project (4th semester)

- Recap of DB design process
- Recap of ORM
- Case study: designing a database for a real-world scenario

Semester Schedule

Session	Date	Time	Topics
1	09.09	12:30–15:00	Introduction; Course overview; Exam info; DB Management System; Data Models
2	15.09	14:00–17:15	Relational Databases; Basic Relational Algebra
3	19.09	09:00–12:15	Advanced Relational Algebra; Normal Forms (1NF, 2NF, 3NF)
4	26.09	09:00–12:15	Normal Forms (BCNF, 4NF, 5NF, 6NF)
5	29.09	14:00–17:15	ERM Design
6	13.10	14:00–17:15	SQL (DML)
7	20.10	14:00–17:15	SQL (DML)
8	27.10	14:00–17:15	SQL (DCL); CAP-Theorem
9	03.11	14:00–17:15	Backup & Recovery; Performance; Concurrency
10	10.11	14:00–17:15	Security; Object-Relational Mapping
11	17.11	14:00–17:15	Exam preparation
12	26.11	10:00–11:00	**Written Exam**

The schedule is designed to move from abstract theory (data models, relational algebra, normal forms) towards increasingly practical content (SQL, ORM, security). Attendance is strongly recommended throughout as later sessions build directly on concepts introduced in earlier ones.

Organization

Mailing List & Contact

- All course announcements will be sent via discord.
- There is no mailing list in place for this class. Please check the discord channel regularly for updates.
- Lecturer contact: `mathis.neunzig@gmail.com` +49 174 9885992

Emergency Contact

If you have an urgent issue that cannot wait for the next session – for example a technical problem on the day of an assignment submission – please use the phone number above. For non-urgent matters, emailor discord is always preferred.

Rules

General Conduct

Standard DHBW rules of conduct apply throughout all sessions.

- **Absences:** Please give advance notice if you cannot attend. A quick message the day before is always appreciated. If you miss a session, it is your responsibility to catch up on the material.
- **Deadlines:** If you foresee that you cannot meet an assignment deadline, contact the lecturer *before* the deadline, not after. Extensions can often be arranged if asked in advance.
- **Open door:** Feel free to reach out at any time with questions, concerns, or feedback; about the content, the pace, or anything else. There are no silly questions in this course.
- **Group exercises:** Active participation during group exercises is expected and contributes to your own learning as well as the group's. Sitting passively during a group task is not acceptable.

These rules are based on mutual respect. The expectation is not perfection but honest communication. Ifsomething is not working for you, say so early rather than letting it become a bigger problem.

Assessment: Written Exam

Key Facts

- **Weight:** 40 points (40 % of final grade)
- **Timing:** End of 3rd semester (12, 26.11)
- **Duration:** 60 minutes

Exam Scope

Topics marked with **(!)** in the slides will *not* appear in the written exam. These are advanced or supplementary topics included for completeness and professional context, but you will not be tested on them under exam conditions.

Examples

Permitted Aids You may bring **one handwritten A4 sheet** (both sides allowed) as a personal cheat sheet. It must be handwritten – printed or typed sheets are not permitted. Writing your own cheat sheet is itself an effective revision strategy.

The written exam tests your understanding of foundational database concepts, relational algebra, normalisation theory, and SQL. Focus on understanding the *why* behind each concept, not just memorising syntax – many exam questions

Assessment: Assignments and Project

Overview – 60 points total (60 % of final grade)

Assignment	Description	Points
Semester 3		
Assignment 1	Individual work	10 pts
Assignment 2	Individual work	10 pts
Semester 4		
Project	Final project (in groups)	40 pts

Submission Policy (Assignments 1 and 2)

- **1st submission** receives a grade *and* written feedback.
- **Optional 2nd submission** is possible after reviewing the feedback.
- The 2nd submission grade replaces the 1st – use this opportunity wisely.

The two-submission model is designed to support your learning: you are encouraged to engage with the feedback and genuinely improve your solution, not just tweak minor details. Assignment 4 is the most substantial deliverable and will give you the opportunity to apply everything you have learned in a realistic

Literature

No textbook purchase is required to pass this course.

All essential material is covered in the lecture slides and exercises.

Recommended reading (for those who want to go deeper):

- Kemper & Eickler – *Datenbanksysteme*
  The German-language standard reference for database courses at German universities. Thorough and mathematically precise.
- Garcia-Molina, Ullman & Widom – *Database Systems: The Complete Book*, 2nd ed.
  Comprehensive English-language textbook covering theory and practice in depth.
- Foster & Godbole – *Database Systems: A Pragmatic Approach*, 2nd ed.
  A more application-oriented introduction, well-suited as a companion to this course.
- Gumm & Sommer – *Einführung in die Informatik*, 10th ed.
  Broader CS introduction, useful for contextualising databases within the wider field.

Reading around the subject – even dipping into one or two chapters – significantly deepens your understanding and will help you engage more confidently with the material during lectures.

Table of Contents

Why Do We Need Data Storage?

Open Question – Think Before Reading On

Why does virtually every software system need some form of persistent data storage?

A First Intuition

- Every meaningful application manages **state** – users, orders, messages, sensor readings, ...
- Without structured storage, **all data is lost** when the application restarts or the machine shuts down.
- Even a mobile flashlight app stores a preference ("was the torch on?").

It is easy to underestimate just how universal data persistence is. From the smallest IoT sensor logging temperature readings to a global e-commerce platform tracking billions of transactions, the need to store, retrieve, and update data reliably sits at the heart of almost all software engineering. Understanding *how* to do this well – efficiently, consistently, and securely – is therefore one of the most fundamental skills in the field. This course gives you the theoretical foundation

The Problem with File-Based Storage

Examples

Scenario: A Supermarket's "Database" The accounting team stores everything in Excel:

- `Customers.xlsx`
- `Items.xlsx`
- `Orders.xlsx`
- `Prices.xlsx`
- `Old_Orders.xlsx`
- `Prices_from_September_1st.xlsx`
- `Urgent_Orders_copy.xlsx`
- `Items_discontinued.xlsx`

Core Problems

- **Ownership silos:** Files belong to different teams (Accounting, HR, Customer Relations, Sales) – no single source of truth.
- **Redundancy:** The same customer address may appear in three files, each with a different spelling.
- **Inconsistency:** After a price update, only some files are changed – leading to contradictory records.

The Need for a Database

Examples

Scenario: Launching "Cat Shop 24" Your team is building a mobile shopping app with:

- Personalised **coupons** and **reward points**
- Monthly **prize draws** for registered users
- Structured **data collection** for analytics

The Core Constraint

A mobile phone **cannot** connect directly to a local Excel file on someone's office PC.

The app needs a server-side data store that is:

- Always available (24/7)
- Accessible over the network
- Capable of handling many users simultaneously
- Consistent and reliable even under load

Data

Definition

Data consists of raw, unprocessed facts. On their own, individual data points carry no direct value or meaning – they are simply observations or records of events.

Examples of raw data:

- A single receipt number: `REC-00482`
- A temperature reading: `21.4`
- A timestamp: `2026-09-09 12:47:33`
- The number: `488000000`

Examples

Analogy: Raw Ingredients Data is like raw ingredients in a kitchen – flour, eggs, sugar – before any cooking has happened. Each ingredient exists, but without a recipe and a chef, nothing useful is produced.

The distinction between data and information might seem philosophical, but it has practical consequences for how we design systems. A database stores data; the application layer (or an analytical query) is responsible for turning that data into

Information

Definition

Information is data that has been processed and contextualised in a way that gives it value and meaning. Information answers a question that someone actually cares about.

Example: A Sales Dashboard

- Raw data: millions of individual transaction records in a database.
- Information derived from that data:
  - *Current year total sales: $488 M*
  - *Budget variance: +3.2 %*
  - *Year-over-year growth: +11.7 %*
  - A chart showing weekly sales trends over 12 months

Examples

The Key Question Data becomes information when it **answers a question**: "How are we performing relative to last year?"

This transformation – from data to information – is one of the core value propositions of database systems. A well-designed database does not just store

Metadata

Definition

Metadata is data *about* data. It provides the structural and administrative context that makes raw data interpretable and manageable.

Types of metadata in a database context:

- **Data types:** `String`, `Integer`, `Boolean`, `Date`, ...
- **Semantic types:** `EmailAddress`, `PhoneNumber`, `PostalCode`, ...
- **Relationships:** "Column `customer_id` references table `Customers`"
- **Administrative data:** `created_at`, `created_by`, `last_modified`, `owner`
- **Access control:** Who may read, write, or delete each piece of data?
- **Derived rules:** Mathematical formulas, computed columns, check constraints

Why Metadata Matters

Without metadata, a column containing the value `21.4` could be a temperature, a price, a probability, or a latitude. Metadata tells us – and the DBMS – what the value *means* and how it may be used.

Metadata is what transforms a flat file full of numbers into a structured, self-describing database. One of the key advantages of using a DBMS over storing

What is a Database System?

Conceptual Overview

$$\textbf{Data} + \textbf{Metadata} \longrightarrow \textbf{Database System} \longrightarrow \textbf{Information}$$

- A **Database System (DBS)** combines:
  - The **DBMS** (Database Management System) – the software layer
  - The actual **stored data** (the database itself)
- Together they form a complete system capable of storing, organising, and serving data reliably.

Examples

Common Database Systems

Relational:	MySQL, PostgreSQL, MS SQL Server, Oracle
Document:	MongoDB, CouchDB
Key-Value:	Redis, DynamoDB
Graph:	Neo4j

Database Management System (DBMS)

What is a DBMS?

A **DBMS** is a software system (informally: "the database") that provides a controlled, structured interface between users/applications and the raw data stored on disk.

Core functions of a DBMS:

1. **Define** data types, table structures, and integrity constraints
2. **Store** data efficiently and without inconsistencies
3. **Retrieve / Query** data in flexible and performant ways
4. **Update** existing data while preserving consistency
5. **Manage concurrent access** by multiple users simultaneously
6. **Protect data** from hardware failures and malicious actors

Additional Components Typically Included

- **User interface:** administration console, query editor
- **Query language:** SQL (for relational systems)
- **Metadata storage:** types, relations, `created_at`, `created_by`, access rights, semantic type annotations

DBMS: Pros and Cons

Advantages

- **Reduced redundancy** – a single source of truth (when designed correctly)
- **Access restriction** – fine-grained permissions per user or role
- **Automatic backup & recovery** – data is protected from failures
- **Multiple user interfaces** – CLI, GUI, programmatic APIs
- **Triggers & automation** – the DBMS can react to events automatically
- **Efficient data processing** – indexes, query optimisers, caching
- . . . and much more

Disadvantages

- **High initial investment:** installation, configuration, licences, hardware
- **Ongoing costs:** administration, tuning, DBA staffing, software maintenance
- **Overkill for trivial tasks:** a config file or a single CSV may be perfectly adequate for a small, single-user application

The decision to adopt a DBMS is ultimately an engineering trade-off. For any

Table of Contents

Data Models & Database Types – Overview

Central Questions

- What data models exist, and why were they invented?
- What are the conceptual differences between them?
- When should you choose one over another?

Data models did not appear all at once – they *evolved* in direct response to real-world problems: growing data volumes, the need for concurrent access, increasingly complex relationships between entities, and the demand for faster, more expressive queries.

Each model represents a different trade-off between **simplicity**, **flexibility**, and **scalability**. Understanding those trade-offs is the key skill for any database designer or software engineer: the "best" model depends entirely on your use case, your data's structure, and your query patterns.

Paper-Based Data Model

Historical Examples

- **Census books** (~18th century) – hand-written population records
- **Marriage registers** (~19th century) – church and civil records of life events
- **Card catalogues** (19th–20th century) – library index cards, one card per book

Paper-based record-keeping served humanity for centuries, but it has severe structural limitations. Finding information required physically searching through stacks of books or boxes of cards – there was no concept of a "query", no possibility of concurrent access, and no means of automation.

Breaking Point: 1890 US Census

The 1890 census had to count **62,947,714** people. Storing and tallying that data manually was no longer feasible – results from the previous census were not fully compiled before the next census began. This crisis forced the invention of mechanical data storage and processing.

Herman Hollerith (!)

Mechanical Punch Card Tabulation – 1890

- Herman Hollerith invented the **punch card reader** and **tabulating machine** specifically for the 1890 US census.
- Process: an operator feeds punch cards through the reader $\rightarrow$ the machine counts holes mechanically $\rightarrow$ produces numerical totals.

Examples

Example query: *"How many people in Baltimore are over 50 years old?"*
With Hollerith's machine this could be answered **mechanically in hours** instead of years of manual counting.

Hollerith's invention was transformative not only for data processing but also for the computing industry itself. The company he founded to commercialise his tabulating machines eventually merged with others to become the **Computing-Tabulating-Recording Company**, which was renamed **IBM** in 1924. Punch cards remained a primary data-entry medium all the way into the 1970s.

(!) This topic is not part of the exam, but its historical significance justifies a brief discussion.

File-Based Data Model

Timeline: 1950s – 1960s

- First **electronic files** stored on magnetic tape and, later, disk
- Data is structured by the *application*: a file on a filesystem with a format the program understands (fixed-width columns, delimiters, etc.)
- Still in everyday use for simple cases: CSV exports, JSON/XML configuration, log files

The file-based model was a massive leap over paper – data could now be read, written, and processed at electronic speed. However, every application had to implement its own logic for reading and interpreting files. There was no shared schema, no query language, and no access control built in.

Key Limitations

- No concurrent access management – two programs writing simultaneously corrupt the file.
- No unified query language – every developer invents their own search.
- Application-specific interpretation – changing the format breaks every program that reads the file.

Hierarchical Data Model

Timeline: 1960s – Developed at IBM

- Data is arranged in a **tree structure**: one root node, parent nodes, and leaf nodes.
- Each record node has *n* fields; each field has exactly **1 value**.
- Each child has **exactly 1 parent**; a parent can have *n* children.

Examples

`Supermarket` → `Food`, `Non-Food`
`Food` → `Cheese`, `Bread`
`Cheese` → `Price`, `Weight`, ...

The hierarchical model introduced the concept of *structural relationships* between data records. Navigating from parent to child is very efficient because the physical pointers are stored alongside the data. This made it well-suited for *predefined, fixed* traversal paths.

Limitation

The strict one-parent rule means **many-to-many relationships cannot be**

IBM Information Management System (IMS) (!)

Timeline: 1966 by IBM

- Originally developed to manage the **Bill of Materials (BOM)** for the Apollo moon program – tracking hundreds of thousands of spacecraft components and their sub-components.
- Based on the **hierarchical data model**; processes one dataset at a time (no set-based operations).
- Programming interfaces: Assembly, C, C++, COBOL, FORTRAN.

IMS is one of the oldest database management systems in continuous active use – it has been maintained and extended by IBM for over 50 years. Large financial institutions (banks, insurance companies) rely on IMS today because their core transaction processing was built on it decades ago and the cost of migration exceeds the cost of continued operation. This is a powerful real-world lesson: choosing a data model has *long-term consequences*.

(!) This topic is not part of the exam.

Network-Like Data Model

Timeline: 1960s – Charles Bachman at General Electric

- Data is represented as a **graph**: *nodes* (records) connected by *arcs* (relationships).
- Unlike the hierarchical model: a child node can have **multiple parents**.
- **Cycles** in the graph are allowed.

Examples

`Supermarket` ↔ `Food` ↔ `Orders` ↔ `Cheese` / `Bread`

A single `Cheese` node can be linked to both `Food` and `Gourmet` categories simultaneously.

The network model was a direct solution to the hierarchical model's many-to-many limitation. By allowing a node to have multiple parents, it can faithfully represent real-world entities – for example, a product that belongs to several categories, or an employee who reports to multiple managers. The price for this expressiveness is **navigational complexity**: programmers had to write explicit code to traverse pointers, which made queries tightly coupled to the physical data structure.

Integrated Data Store (IDS) (!)

Timeline: 1964 by General Electric – Charles Bachman

- One of the **first true Database Management Systems (DBMS)** in history.
- Implemented the network-like data model with physical pointer-based navigation between records.
- Became the basis for the **CODASYL** (Conference on Data Systems Languages) **Data Base Task Group** standard – the first attempt to standardise a DBMS interface across vendors.

IDS demonstrated that a general-purpose data management layer could exist independently of any single application – a fundamental idea that all subsequent database systems inherited. Charles Bachman received the **Turing Award** in 1973 for his work on IDS and database navigation, making him the first database researcher to be so recognised.

The CODASYL standard eventually influenced the design of COBOL's data management facilities, which is why so much legacy financial software still bears traces of the network model's thinking.

(!) This topic is not part of the exam.

Relational Data Model

Timeline: 1969 by Edgar F. Codd (IBM)

- Data stored as **tuples** (rows); tuples grouped into **relations** (tables).
- **Keys** uniquely identify tuples within a relation.
- **Closure property**: the result of any operation on relations is itself a relation – queries can be composed freely.

Examples

`Person(PersonID, FirstName, LastName, Address)`
`Order(OrderID, Date, PersonID`$^{\text{FK}}$`)`

Codd's insight was to *separate the logical view of data from its physical storage*. Applications should describe *what* data they need, not *how* to navigate to it. A high-level, set-based query language (eventually SQL) would handle the navigation automatically. This **data independence** made it possible to change storage structures without rewriting application code – a revolutionary idea at the time. The relational model became the dominant paradigm through the 1980s and remains the most widely used database model today.

The Rise of RDBMS in the 1980s

Competing Implementations

- **UC Berkeley**: Ingres project → eventually became **PostgreSQL** (still one of the most popular open-source RDBMSs)
- **Uppsala University**: research system influencing European DBMS development
- **IBM UK & IBM San José**: System R project → gave birth to the **SEQUEL** query language, later renamed **SQL**

Throughout the late 1970s and 1980s, academic labs and IBM research centres raced to build working relational systems. IBM had the research lead but was slow to productise – partly because they feared cannibalising IMS sales. **Oracle** (then called Relational Software, Inc.) recognised the commercial opportunity, shipped the first commercial SQL RDBMS in **1979**, and never looked back.

Key Lesson

Being technically first does not always mean winning the market. Oracle's aggressive go-to-market strategy beat IBM's superior research effort – a pattern that repeats throughout technology history.

MySQL

Timeline: 1995 by MySQL AB (now Oracle)

- Open-source, cross-platform, SQL-based relational database.
- **Fun fact**: "My" is the name of co-founder Michael Widenius's daughter.
- Famous for powering the **LAMP / WAMP stack**:
  Linux (or Windows) + Apache + MySQL + PHP.

MySQL's release in 1995 democratised relational databases. Before MySQL, a production-quality SQL database required an expensive commercial licence (Oracle, Sybase, Informix). MySQL was free, fast enough for web workloads, and trivial to install – making it the default choice for the first generation of web developers and powering much of the early internet.

Oracle acquired Sun Microsystems (which had acquired MySQL AB) in 2010. Concern about Oracle's stewardship led the original MySQL developers to fork the codebase and create **MariaDB** – the community-maintained drop-in replacement that is now the default in many Linux distributions.

XAMPP

Timeline: 2002 by Apache Friends

- **X** = Cross-platform (Windows, macOS, Linux)
- **A** = Apache HTTP Server
- **M** = MariaDB (formerly MySQL)
- **P** = PHP
- **P** = Perl

XAMPP bundles a complete local web-and-database server stack into a single installer. For learning purposes it is ideal: one download gives you a running web server, a relational database, and a scripting runtime without any manual configuration. It ships with **phpMyAdmin**, a browser-based graphical administration tool for MariaDB/MySQL.

Examples

In this course we use XAMPP to host a local MariaDB instance. You can reach phpMyAdmin at `http://localhost/phpmyadmin` after starting the Apache and MySQL/MariaDB services in the XAMPP Control Panel.

Object-Oriented Data Model

Timeline: 1980s

- Maps **object-oriented programming** concepts directly to database storage.
- Relation ≡ Class Tuple ≡ Instance Column ≡ Attribute Stored Procedure ≡ Method
- Gave rise to semi-structured formats such as **XML** and **JSON** for data exchange and persistence.

Examples

An `Item` class with attributes `name`, `price`, `category` can be serialised directly to JSON:

```
{"name":"Gouda","price":3.49,"category":"Cheese"}
```

The object-oriented model emerged because developers found the *impedance mismatch* between object-oriented code and relational tables painful – every object had to be manually mapped to rows and columns. Object-oriented databases promised to eliminate this friction by persisting objects natively. Adoption was limited because relational systems were already entrenched, but the ideas strongly influenced ORM frameworks (Hibernate, SQLAlchemy, Django ORM) which

ObjectStore (!)

Timeline: 1988 by Versant (originally Progress Software)

- A database whose unit of storage is a **C++ object**.
- Calling `new()` in C++ creates a **persistent object** that survives process termination – no separate "save" step.
- Use cases: telecommunications, Geographic Information Systems (GIS), financial trading systems.

ObjectStore showed that the boundary between "in-memory data structures" and "persistent database records" could be made nearly invisible to the programmer. The system handled serialisation, indexing, and concurrent access automatically. This concept re-appears in modern object-document mappers and in-memory databases like Redis.

ObjectStore is still in active use in niche domains where the original object-oriented codebase is too large and too critical to replace.

(!) This topic is not part of the exam.

1990s – The Quiet Decade

Consolidation Around the Relational Model

- No major new database paradigm emerged in the 1990s.
- SQL became an ISO/ANSI standard (SQL-89, SQL-92, SQL:1999).
- Significant advances in **data warehousing** and **analytical processing** (OLAP, star schemas, data marts).
- MySQL (1995) made relational databases accessible to web developers.

The 1990s were important not because of new data models but because of *maturation*. Query optimisers became sophisticated enough that SQL performance rivalled hand-tuned navigational code. Replication, backup, and transaction management were standardised. The internet boom at the end of the decade set the stage for the next wave of innovation – the *NoSQL* movement – which was driven by internet-scale data volumes that relational systems struggled to handle.

Columnar Data Model

Timeline: 2000s – Apache Cassandra, HBase, Parquet

- In a **row-based** store (traditional RDBMS), each row is stored contiguously on disk.
- In a **columnar** store, each *column* is stored contiguously – all values for one attribute sit together.
- Reading an analytical query touching only 3 of 100 columns means reading ≈3% of the data instead of 100%.

Examples

`SELECT AVG(price) FROM products WHERE category = 'Electronics'`
Only the `price` and `category` columns need to be read from disk – all other columns are skipped entirely.

Columnar storage also enables **exceptional compression**: because all values in a column share the same data type, techniques like run-length encoding and dictionary encoding achieve very high compression ratios. The trade-off is that *writing* a single new row requires updating many separate column files, making columnar stores slower for transactional (OLTP) workloads. They shine in

Apache Parquet (!)

Timeline: 2013 – Apache (Twitter & Cloudera)

- A **columnar file format** for big-data processing (Apache Spark, Hadoop, Hive, Presto).
- Often described as a "Big Data File Format": data is spread across multiple Parquet files rather than a single monolithic file.
- Internal structure: **Header** $\rightarrow$ Row Group 1 $\rightarrow$ Row Group 2 $\rightarrow \cdots \rightarrow$ **Footer**
- Within each Row Group, column data is stored contiguously, enabling efficient compression and selective column reads.

Parquet has become the de-facto standard for storing analytical datasets in cloud data lakes (AWS S3, Azure Data Lake, Google Cloud Storage). Its tight integration with Apache Arrow enables zero-copy reads directly into in-memory data frames (pandas, Polars), making the transition from storage to computation extremely fast.

(!) This topic is not part of the exam.

Key-Value Data Model

Timeline: Concept from late 1970s; mainstream from 2000s

- The **simplest NoSQL model**: every entry is a *key* $\mapsto$ *value* pair.
- The value can be anything: a string, an integer, a binary blob, a serialised object – the database does not interpret it.
- **No schema**, no relationships, retrieval only by exact key.

Examples

`"session:u42f8"` $\mapsto$ `"{userId:7, cart:[3,11,42]}"`
`"rate_limit:192.168.1.5"` $\mapsto$ `"47"`

The key-value model's strength is raw speed: lookup by key is an O(1) hash-table operation. This makes it perfect for **caches**, **session stores**, and **temporary logging**.

Key Limitation

You can only retrieve data by its exact key. There is no way to ask "find all sessions belonging to users from Germany" without scanning every entry – because the database has no knowledge of the value's internal structure. For

Redis

Timeline: 2009 by Salvatore Sanfilippo

- The **most widely used key-value database** in the world.
- **In-memory** store: all data lives in RAM $\rightarrow$ sub-millisecond read/write latency.
- Optional **persistence** to disk (RDB snapshots, AOF log).
- Supports richer value types beyond plain strings: Lists, Sets, Sorted Sets, Hashes, Streams, HyperLogLogs, Bitmaps.

Redis is the backbone of many internet-scale architectures. Common roles:

- **Cache layer** in front of a relational database
- **Session store** for stateless web applications
- **Pub/Sub message broker** between microservices
- **Rate limiter** (atomic increment on a counter key)
- **Leaderboard** (Sorted Set with score-based ranking)

A free hosted tier is available at `redis.io`; the source code is open-source on GitHub.

Document-Based Data Model

Timeline: Popular from 2000s onward

- A **specialised key-value store** where the *value* is a structured **document** (JSON, BSON, or XML).
- Documents are grouped into **collections**; no fixed schema – different documents in the same collection can have different fields.
- Unlike plain key-value, you can **query by any field** inside the document.

Examples

Document 1: `{name:"Alice", email:"a@x.com"}`
Document 2: `{name:"Bob", email:"b@x.com", phone:"0711...", address:{...}}`

Both documents live in the same `users` collection despite having different shapes.

This **schema flexibility** makes document databases ideal for rapidly evolving data (e.g. product catalogues where different product types have completely different attributes) or for aggregating data from many sources. The trade-off is that consistency enforcement must be handled in application code rather than by the database schema.

MongoDB

Timeline: 2009 by 10gen (now MongoDB Inc.)

- Stores documents in **BSON** (Binary JSON) format – a binary-encoded extension of JSON that supports additional types (dates, binary data, ObjectId).
- **Hierarchy**: Document $\subset$ Collection $\subset$ Database $\subset$ Cluster.
- Drivers available for all major programming languages (Python, Java, Node.js, C#, Go, PHP, ...).
- Free tier available at `mongodb.com` (MongoDB Atlas).

MongoDB popularised the document model and the term "NoSQL" in the mainstream developer community. It introduced an expressive query language (MQL – MongoDB Query Language) that operates on nested JSON structures, aggregation pipelines for complex data transformations, and a flexible indexing system.

Common use cases: content management systems, product catalogues, user-profile stores, real-time analytics, and mobile application backends.

Graph-Based Data Model

Timeline: 1980s evolution of network model; mainstream 1990s–2000s

- Data represented as **Nodes** (entities with properties) connected by **Edges** (relationships with properties).
- Both nodes *and* edges can carry arbitrary key-value properties.
- Queries traverse the graph to follow relationship chains of arbitrary depth.

Examples

Social network:
`(Alice)-[:FRIENDS_WITH]->(Bob)-[:FRIENDS_WITH]->(Carol)`
"Find all friends-of-friends of Alice who live in Stuttgart" is a natural graph traversal, but painful in SQL (recursive CTEs or multiple self-joins).

Graph databases excel when **relationship traversal** is the dominant access pattern: social networks, fraud detection (rings of suspicious accounts), knowledge graphs, recommendation engines, and network/IT infrastructure mapping. In a relational database, deep traversals require expensive recursive joins; in a graph database they are first-class operations.

Neo4j

Timeline: 2010 by Neo Technology

- Query language: **Cypher** – a pattern-matching language designed to be human-readable (ASCII-art graph patterns).
- Components: **Nodes** (labelled circles), **Edges** (directed, typed relationships), **Properties** (key-value pairs on both).
- Industries: insurance, banking, retail, life sciences, cybersecurity.

Examples

```
MATCH (p:Person)-[:BOUGHT]->(i:Item) RETURN p, i
```

This returns every Person node and the Item nodes they bought, by following `BOUGHT` edges – equivalent to a SQL JOIN but expressed as a graph pattern.

Neo4j is the most widely deployed graph database. Its native graph storage engine stores adjacency lists directly on disk, making multi-hop traversals orders of magnitude faster than relational equivalents. A free "AuraDB" cloud tier is available for experimentation.

Summary: Data Models Comparison

Comparison Table

Data Model	Stored As	Simple	Flexible	Scalable
Paper / File based	Text	✓	✓	×
Hierarchical	Nodes & Arcs	✓	×	×
Network-like	Nodes & Arcs	✓	×	×
Relational	Tables	∼	∼	∼
Object-Oriented	Objects	∼	∼	∼
Columnar	Table / Columns	∼	×	✓
Key-Value	Key-Value Pairs	✓	✓	✓
Document	Documents	✓	✓	✓
Graph	Nodes & Edges	∼	✓	✓

Important Caveat

All ratings above are generalisations – the real answer is always **"it depends on the use case"**. A relational database can scale very well with proper indexing and sharding; a document database can be inflexible if you force a rigid schema on it.

Rule of Thumb

Choose your data model based on three factors: **(1)** the natural structure of your data, **(2)** the query patterns your application will execute most frequently, and **(3)** your scalability and consistency requirements. No single model is universally best.

Table of Contents

Edgar Frank "Ted" Codd

Biographical Overview

- **19 August 1923 – 18 April 2003**
- Inventor of the **relational data model**, the concept of **OLAP** (Online Analytical Processing), and co-designer of **SEQUEL** (the precursor to SQL).

Career Timeline at IBM

- **1949**: Joins IBM New York as a developer.
- **1965**: PhD in computer science, University of Michigan (funded by IBM scholarship).
- **1967**: Moves to IBM Almaden Research Center, San José.
- **1969–1970**: Publishes the foundational papers on the relational model, normal forms, and a relational sub-language.
- **1970s**: Co-develops SEQUEL with Donald Chamberlin.

Codd's work matters because **almost all modern business software** – banking, e-commerce, ERP, CRM, logistics – runs on relational databases that implement his mathematical framework. He won the **Turing Award** in 1981 for his

Relations

Definition

A **relation** is a mathematical set of tuples that all share the same structure (schema). It can be visualised as a **table** with named columns and a collection of rows.

- Columns = **Attributes** (named, typed)
- Rows = **Tuples** (concrete data records)
- A table contains $0 \ldots *$ tuples.

Examples

`Album(`<u>`ID`</u>`, EAN, Title, Main_Artist, Release_Date)`
`Track(`<u>`ID`</u>`, Album_ID`$^{\text{FK}}$`, Disc_Nr, Track_Nr, ISRC, Title, Duration)`

The word *relation* comes from the Latin *relatio* – the relationship between entities of the same kind. Because a relation is a **mathematical set**, it contains *no duplicate tuples* in the pure theoretical definition. In practice, SQL allows duplicates unless you explicitly use `DISTINCT` or enforce a uniqueness constraint – an important distinction between the theory and its implementation.

The **closure property** of the relational model states that the result of any

Attributes

Definition

An **attribute** is a named column in a relation that defines one specific piece of data about the entities stored in that relation. Every attribute has:

- A **name** (e.g. `Release_Date`)
- A **domain** – the set of all allowed values (e.g. `DATE`, `VARCHAR(255)`, `INTEGER`)

Examples

Attributes of `Album`: `ID`, `EAN`, `Title`, `Main_Artist`, `Release_Date`

The domain of `EAN` is a 13-digit integer; the domain of `Title` is a non-empty string of up to 255 characters.

Attributes describe the *properties* or *characteristics* of the entities stored in a table. Choosing good attribute names is not a minor stylistic concern – unclear names lead to misunderstandings, bugs, and costly data migrations later. Best practices include:

- Use **self-describing** names (`release_date` not `rd` or `date1`).
- Apply a **consistent naming convention** across the entire schema

Tuples

Definition

A **tuple** is a single row (record) in a relation – an ordered list of values, one per attribute, all conforming to their respective domains. A tuple may contain `NULL` for optional attributes.

Examples

A tuple of `Album`:

```
(K5MIF8A, 4130971072345, Fearless, Taylor Swift, 2008-11-11)
```

A tuple of `Track`:

```
(T00042, K5MIF8A, 1, 1, USUG10801234, Love Story, 00:03:55)
```

The term *tuple* comes from mathematics (a finite ordered list: pair, triple, quadruple, ..., n-tuple). In relational theory each tuple is a complete, self-contained fact about an entity. In SQL terminology a tuple is a **row**; in object-oriented terminology it corresponds approximately to an **instance** of a class.

A crucial point: within one relation, **no two tuples may be identical** in the strict relational model. This is why every table needs at least one key – to distinguish

Types of Keys

Overview

Keys serve two purposes in the relational model:

1. **Identification**: uniquely identify every tuple within a relation.
2. **Reference**: establish relationships between tuples in different relations.

Key Taxonomy

- **Super Key** – any set of attributes that uniquely identifies tuples.
- **Candidate Key** – a minimal super key.
- **Primary Key** – the chosen candidate key for a relation.
- **Alternate / Secondary Key** – candidate keys not chosen as primary key.
- **Foreign Key** – an attribute referencing a primary key in another relation.
- **Simple Key** – a key consisting of a single attribute.
- **Composite Key** – a key consisting of two or more attributes.
- **Natural Key** – a key derived from a real-world attribute.
- **Surrogate Key** – an artificially generated identifier.

Super Keys

Definition

A **super key** is *any* set of one or more attributes whose combined values are unique across all tuples in a relation.

- A super key may contain *more attributes than necessary* for uniqueness – it can include redundant attributes.
- Every relation has at least one super key: the set of *all* its attributes is always a super key (trivially).

Examples

Super keys of `Album(ID, EAN, Title, Main_Artist, Release_Date)`:

`{ID}`, `{EAN}`, `{ID, EAN}`, `{ID, Title}`, `{ID, EAN, Title}`, ...

Even `{ID, EAN, Title, Main_Artist, Release_Date}` is a super key.

The concept of a super key is the *starting point* for the key taxonomy. In practice, super keys with redundant attributes are not directly useful – we refine them into candidate keys by removing unnecessary attributes. However, understanding super keys helps clarify *why* minimality matters: smaller keys mean smaller indices,

Candidate Keys

Definition

A **candidate key** is a *minimal* super key: no proper subset of its attributes is also a super key.

- **Unique**: no two tuples share the same candidate key value.
- **Irreducible**: removing any attribute from the key breaks uniqueness.
- A relation can have **multiple** candidate keys.

Examples

Candidate keys of `Album`:

- `ID` – internal surrogate identifier (UUID or integer)
- `EAN` – 13-digit product barcode (unique per album)

Candidate keys of `Track`:

- `ID` – surrogate key
- `ISRC` – International Standard Recording Code (unique per recording)
- {`Album_ID`, `Disc_Nr`, `Track_Nr`} – composite natural key

Primary Key

Definition

The **primary key (PK)** is the candidate key officially chosen as the main identifier for tuples in a relation.

- Must be **unique** for every tuple.
- Must **never be** `NULL`.
- Should be **stable** – it should rarely or never change after a tuple is created.

Examples

`Album(ID, EAN, Title, Main_Artist, Release_Date)`
The primary key is `ID`; it is underlined by convention.

In SQL, the `PRIMARY KEY` constraint automatically enforces both uniqueness and `NOT NULL`. Most database systems also create a **clustered index** on the primary key, which physically orders the rows on disk by PK value – making primary-key lookups extremely fast.

Why stability matters

If a primary key changes, every foreign key that references it must also be updated

Alternate / Secondary Keys

Definition

Alternate keys (also called *secondary keys*) are candidate keys that were *not* selected as the primary key. They are still genuine unique identifiers – we simply chose a different candidate key to be the PK.

Examples

In `Album`: Primary key = `ID`, so `EAN` becomes the **alternate key**.

In SQL: `UNIQUE (EAN)` – enforces uniqueness without making `EAN` the primary key.

Alternate keys serve an important practical role: **external systems** often know a natural identifier (EAN, ISBN, ISRC) but not the internal surrogate ID. By enforcing a unique constraint on the alternate key, you allow efficient lookups from those external systems while keeping a clean surrogate PK for internal foreign key references.

A good database schema **enforces all candidate keys**, not just the primary key. Failing to do so allows logically duplicate records that differ only in their surrogate ID – a common source of data quality problems in production systems.

Foreign Keys

Definition

A **foreign key (FK)** is an attribute (or set of attributes) in one relation whose values must match an existing primary key value in another (or the same) relation – or be `NULL`.

- Enforces **referential integrity**: you cannot reference a tuple that does not exist.
- Creates a **parent–child** relationship between tables.

Examples

`Track(`<u>`ID`</u>`,` *`Album_ID`*$^{\texttt{FK}}$`, Disc_Nr, ...)`

`Album_ID` in `Track` references `ID` in `Album`.
Inserting a track for a non-existent album raises an integrity error.

Referential Integrity Actions

When a referenced tuple is deleted or its PK is updated, the database must decide what to do with referencing rows. SQL offers: `RESTRICT`, `CASCADE`, `SET NULL`, `SET DEFAULT`. Choosing the right action is part of schema design – `CASCADE`

Simple vs. Composite Keys

Simple Key

A **simple key** consists of a **single attribute**.

- Easy to reference as a foreign key (one column).
- Typically a surrogate (auto-increment integer or UUID).

Composite Key

A **composite key** consists of **two or more attributes** whose combined values are unique.

- Common in **junction / association tables** that represent many-to-many relationships.
- Natural uniqueness constraints often emerge as composite keys.

Examples

Simple key: `ID` in `Album`

Composite key: `{Album_ID, Disc_Nr, Track_Nr}` in `Track`
$\Rightarrow$ Disc 1, Track 3 of album "K5MIF8A" identifies exactly one track.

Natural vs. Surrogate Keys

Natural Key

A **natural key** is derived from a real-world attribute that already exists in the data domain.

- Examples: EAN (barcode), ISBN (book), ISRC (music track), social security number, email address.
- **Pros**: human-readable, meaningful, no artificial column needed.
- **Cons**: may change (email addresses change!), often long (bad for indices and FK columns), may be unavailable at insert time.

Surrogate Key

A **surrogate key** is an artificially generated identifier with no inherent business meaning.

- Examples: auto-increment integer (`SERIAL`), UUID v4.
- **Pros**: stable, compact, always available, never changes.
- **Cons**: meaningless on its own, requires joining to see human-readable identity.

Exercise: Relational Databases

Task

Work through relations **Exercise 1A** through **Exercise 1D**.
(Simple: 1A & 1B Intermediate: 1C & 1D)

For each relation, answer the following questions:

1. List all **attributes** and their domains.
2. Identify all **candidate keys**; select and justify your choice of **primary key**.
3. Identify all **foreign keys** and state which primary key each one references.
4. Classify every key as *simple* / *composite* and *natural* / *surrogate*.
5. Create **2 realistic sample tuples** for each relation that respect all constraints.

Time: 15 minutes

Then we will discuss the solutions together. There is often more than one valid choice of primary key – be prepared to justify yours.

Preview: Relational Algebra

Why Do We Need It?

Defining relations, attributes, and keys tells us *how to store* data. We now need a formal language to describe *how to query and manipulate* that data.

Relational algebra provides the mathematical foundation for all relational query languages, including SQL. It defines a small set of **operations on relations that produce new relations** (closure property), which can be freely composed.

Core operations we will cover:

- Selection σ – filter rows by a predicate
- Projection π – select a subset of columns
- Rename ρ – rename a relation or attribute
- Union $\cup$, Intersection $\cap$, Difference $\setminus$ – set operations
- Cartesian product $\times$ – combine every pair of tuples
- Join $\bowtie$ – combine related tuples (natural join)
- Left / Right outer join $\ltimes$ / $\rtimes$
- Division $\div$ – "find all X that are associated with all Y"

Every SQL query you will ever write is an expression in relational algebra in disguise – understanding the algebra makes SQL comprehensible rather than a collection of memorised syntax.

Preview: Normal Forms

Motivation

Not all relation designs are equally good. A poorly designed schema can lead to:

- **Redundancy** – the same fact stored in multiple places.
- **Update anomalies** – changing one fact requires updating many rows; missing one causes inconsistency.
- **Insertion anomalies** – you cannot record a fact without first recording an unrelated fact.
- **Deletion anomalies** – deleting one fact accidentally destroys another.

Normal Forms

Normal forms are a hierarchy of design rules that systematically eliminate these problems by ensuring each fact is stored exactly once. We will cover:

- **1NF** – atomic values, no repeating groups
- **2NF** – no partial functional dependencies on a composite PK
- **3NF** – no transitive functional dependencies
- **BCNF** – Boyce–Codd Normal Form (stronger 3NF)

Table of Contents

What is Relational Algebra?

Relational algebra is a **formal, procedural query language** for relational databases. Unlike declarative languages (where you state *what* you want), relational algebra requires you to specify *how* to retrieve data by composing a sequence of well-defined operations. It forms the mathematical backbone of SQL and every other relational query language.

Core Properties

- Every operation takes one or two *relations* as input and produces exactly one *relation* as output — the **closure property**. This means operations can be composed freely.
- Provides the theoretical foundation for SQL; every SQL `SELECT` statement can be translated into a relational algebra expression.
- By combining operations arbitrarily, queries of any complexity can be expressed.

Named Operators

Union $\cup$, Intersection $\cap$, Difference $\setminus$, Cartesian Product $\times$, Division $\div$, Restriction σ, Projection π, Rename ρ, Join $\bowtie$

Overview of All Operators

The table below lists all operators covered in this lecture. Unary operators act on a single relation; binary operators combine two relations. Join operators are a special family of binary operators and are treated separately in later frames.

Type	Symbol	Operator	Description
Binary	$\cup$	Union	All tuples from both relations, no duplicates
Binary	$\cap$	Intersection	Only tuples common to both
Binary	$\setminus$	Difference	Tuples in R_1 but not in R_2
Binary	$\times$	Cartesian Product	Every combination of tuples
Binary	$\div$	Division	Tuples in R_1 associated with ALL tuples in R_2
Unary	σ	Restriction	Filter rows by condition
Unary	π	Projection	Select specific columns
Unary	ρ	Rename	Rename relation or attributes
Unary	δ	Deduplication	Remove duplicate tuples
Unary	γ	Grouping/Aggregation	Group and aggregate
Unary	τ	Sorting	Sort tuples
Join	$\bowtie$	Join	Combine related tuples from two relations

Union $\cup$

The union operator **merges all tuples** from two relations into a single result relation. Duplicate tuples — tuples that appear in both input relations — are *eliminated automatically*, just as in mathematical set union.

Key Points

- Both relations must have the **same schema** (same number of attributes with compatible domains). This requirement is called *union compatibility*.
- Syntax: $R_1 \cup R_2$
- The result contains *at most* $|R_1| + |R_2|$ tuples; duplicate tuples reduce this count.

Examples

Example **EmployeesA** $\cup$ **EmployeesB** — two employee tables from different departments or branches are combined into one unified list. Every employee appears exactly once, even if a person happened to be recorded in both tables.

Intersection ∩

The intersection operator returns only those tuples that are present in **both** input relations simultaneously. It answers the question "what do these two sets have in common?"

Key Points

- Both relations must be *union-compatible* (same schema).
- Syntax: $R_1 \cap R_2$
- Intersection can be **derived** from the difference operator — it does not have to be a primitive operation:

$$R_1 \cap R_2 \;=\; R_1 \setminus (R_1 \setminus R_2)$$

Examples

Example **Employees** ∩ **Volunteers** — given a relation of all employees and a relation of all people who volunteered for the Christmas party, the result contains exactly the employees who also volunteered. People who volunteered but are not employees are excluded.

Difference $\setminus$

The difference operator returns all tuples that are in R_1 **but not** in R_2. It is sometimes written with a minus sign: $R_1 - R_2$.

Order Matters!

Unlike union and intersection, difference is **not commutative**: $R_1 \setminus R_2 \neq R_2 \setminus R_1$ in general. Swapping the operands gives an entirely different result.

Key Points

- Both relations must be union-compatible.
- Syntax: $R_1 \setminus R_2$
- The **Symmetric Difference** Δ returns tuples that appear in *either* relation but *not both*:

$$\Delta = (R_1 \setminus R_2) \cup (R_2 \setminus R_1)$$

Examples

Examples

- **Employees** $\setminus$ **Volunteers** — employees who did *not* volunteer.
- **Volunteers** $\setminus$ **Employees** — volunteers who are *not* employees (e.g., external

Cartesian Product $\times$

The Cartesian product (also called *cross product*) combines **every** tuple of R_1 with **every** tuple of R_2, regardless of whether the combination is meaningful.

Key Points

- Result size: $|R_1| \times |R_2|$ tuples.
- The result schema contains *all* attributes from both relations (concatenated).
- Rarely useful on its own — it produces the full cross product including many nonsensical or redundant combinations.
- Usually the **precursor to a Join**: a Join is simply a Cartesian Product followed by a Restriction that filters only the meaningful combinations.
- Syntax: $R_1 \times R_2$

Examples

Example **Letters**$(\{A, B, C\}) \times$ **Numbers**$(\{1, 2, 3\})$ yields 9 tuples: $(A, 1), (A, 2), (A, 3), (B, 1), \ldots, (C, 3)$. Without further filtering, all 9 combinations appear in the result.

Restriction σ

The restriction operator (also called *selection*) **filters rows** of a relation, keeping only those tuples that satisfy a given Boolean condition. It is the relational algebra equivalent of the SQL `WHERE` clause.

Key Points

- The *schema* of the output is identical to the input; only the number of rows may decrease.
- Conditions can use comparison operators ($=$, $\neq$, $<$, $>$, $\leq$, $\geq$) and logical connectives $\wedge$ (AND) and $\vee$ (OR).
- Multiple restrictions can be chained or merged:

$$\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_1 \wedge C_2}(R)$$

- Syntax: $\sigma_{\text{condition}}(R)$

Examples

Example

$$\sigma_{\text{Department}=\text{'D01'}}(\text{Employees})$$

Projection π

The projection operator **selects specific columns** from a relation, discarding all other attributes. It corresponds to naming individual columns in a SQL `SELECT` list (as opposed to `SELECT *`).

Key Points

- The number of rows stays the same *unless* removing columns causes duplicate tuples — duplicates are then eliminated (pure relational algebra). In SQL you need `DISTINCT` explicitly.
- Syntax: $\pi_{\text{col}_1, \text{col}_2, \ldots}(R)$

Examples

Examples

- $\pi_{\text{User, ID, Department}}(\text{Employees})$ — return only those three columns from the Employees relation.
- Combining restriction and projection:

$$\pi_{\text{User}}(\sigma_{\text{Dept}=\text{'D01'}}(\text{Employees}))$$

First filter to department D01, then project to the User column only. This is

Rename ρ

The rename operator changes the **name of a relation or its attributes** without modifying the actual data. It is a purely structural operation needed whenever two relations that are to be combined share attribute names that would otherwise clash.

Key Points

- **Rename relation:** $\rho_{\text{NewName}}(R)$ — gives the whole relation a new name.
- **Rename attribute:** $\rho_{(\text{NewAttr}/\text{OldAttr})}(R)$ — renames one attribute; all data remain unchanged.
- Particularly important when self-joining a table (you need two differently named copies) or when joining tables that happen to share a column name that should *not* be the join criterion.

Examples

Examples

- $\rho_{\text{Employees}}(\text{Empl})$ — rename the relation `Empl` to `Employees`.
- $\rho_{(\text{Username}/\text{User})}(\text{Employees})$ — rename the attribute `User` to `Username` inside the Employees relation.

Division $\div$

Division is the most complex basic operator. $R_1 \div R_2$ returns all values of the *non-shared* attributes of R_1 for which the corresponding shared attributes cover **every** tuple in R_2. In plain language: "find all X that are associated with *all* Y."

Formal Definition

Let A denote the attributes in R_1 that are *not* in R_2. Then:

$$R_1 \div R_2 \;=\; \pi_A(R_1) \;\setminus\; \pi_A\big((\pi_A(R_1) \times R_2) \setminus R_1\big)$$

This derivation shows that division is not strictly primitive — it can be expressed using projection, Cartesian product, and difference.

Examples

Example **Families** $\div$ **Children** — suppose the Children relation lists two children: *Linda* and *Haley*. The result contains every parent tuple from Families who has *both* Linda and Haley as children. A parent with only one of the two is excluded.

Additional Unary Operators

Basic relational algebra covers the operators introduced so far, but three additional unary operators are commonly added in practice to make the language more expressive and to bridge the gap to real SQL features.

Duplicate Elimination δ

Removes all duplicate tuples from a relation — equivalent to SQL `DISTINCT`.
Example: $\delta(\pi_{\text{Department}}(\text{Employee}))$ — return a list of distinct department names.

Grouping & Aggregation γ

Groups tuples by one or more attributes and computes aggregate functions (`COUNT`, `SUM`, `AVG`, `MAX`, `MIN`) over each group — equivalent to SQL `GROUP BY`.
Example: $\gamma_{\text{Department, SUM(Salary)}}(\text{Employee})$

Sorting τ

Sorts the tuples of a relation by one or more attributes with optional `ASC`/`DESC` direction — equivalent to SQL `ORDER BY`.
Example: $\tau_{\text{Department, Salary DESC}}(\text{Employee})$

Join $\bowtie$ — Introduction

A **join** combines related tuples from two relations based on a shared condition, typically an attribute that exists (conceptually) in both relations. Joins are by far the most frequently used binary operation in practice because relational databases deliberately split information across multiple tables.

Core Idea

$$R_1 \bowtie_C R_2 \;=\; \sigma_C(R_1 \times R_2)$$

A join is *syntactic sugar* for a Cartesian Product followed by a Restriction. The join condition C filters out the combinations that are not meaningful, leaving only the related pairs.

Examples

Motivating Example A `Products` table stores a `Brand_ID` foreign key; a `Brands` table stores `ID` and `Name`. To display product names alongside brand names, the two tables must be joined on `Products.Brand_ID = Brands.ID`.

Types of Joins

There are several join variants, each handling *non-matching* tuples differently. The table below gives an overview; subsequent frames discuss the most important ones in detail.

Symbol	Name	Description
$\bowtie$	Natural Join	Join on all attributes with the same name
$\bowtie_C$	Inner / Theta Join	Join on specified condition
$\ltimes$	Left Semi Join	Left tuples with a match (left attrs only)
$\rtimes$	Right Semi Join	Right tuples with a match (right attrs only)
$\bowtie_L$	Left Outer Join	All left tuples; NULLs for non-matches
$\bowtie_R$	Right Outer Join	All right tuples; NULLs for non-matches
$\bowtie_{LR}$	Full Outer Join	All tuples from both sides
$\triangleright$	Anti Join	Left tuples with *no* match in right

Natural Join $\bowtie$

The natural join **automatically** identifies all attributes that share the same name in both relations and joins on the equality of those attributes. In the result, the shared attribute columns appear only once (duplicates are removed).

Key Points

- No explicit join condition needs to be specified — the common attribute names define it implicitly.
- **Danger:** if two tables coincidentally share an attribute name that should *not* be part of the join condition, the natural join silently produces wrong results. Always check which attributes are shared before using it.
- Syntax: $R_1 \bowtie R_2$
- Can be chained: $A \bowtie (X \bowtie M)$ joins three tables in sequence.

Examples

Example Relations A and X both have an attribute called `ID`. Then $A \bowtie X$ joins them on $A.\texttt{ID} = X.\texttt{ID}$, and the result contains `ID` as a single column.

Inner Join / Theta Join $\bowtie_C$

The **inner join** (equi-join) joins two relations on an *explicitly specified* equality condition. The more general **theta join** allows any comparison operator ($=$, $<$, $>$, $\leq$, $\geq$, $\neq$) in the condition.

Key Points

- Only tuples that have a *matching counterpart* in the other relation appear in the result. Non-matching tuples are discarded — this is the defining characteristic of an inner join.
- The result schema contains all attributes from both relations (unlike the natural join, attribute names are not merged).
- Syntax: $R_1 \bowtie_{R_1.A = R_2.B} R_2$

Examples

Example

$$\text{Products} \bowtie_{\text{Products.Brand\_ID} = \text{Brands.ID}} \text{Brands}$$

Returns all products together with their corresponding brand data. Products without a matching brand entry and brands without any product are both excluded from the result.

Combining Operators

The true power of relational algebra lies in **composing** operators freely. Because every operator returns a relation, the output of any operation can immediately become the input of another.

Reading Strategy

Relational algebra expressions are read **inside-out**: start from the innermost expression and work outward, applying each successive operator to the result of the previous one.

Examples

Example: Product Name and Brand Name Only

$$\pi_{\text{Products.ID, Product\_Name, Brand\_Name}}\Big(\rho_{\text{Product\_Name/Name}}(\text{Products}) \bowtie_{\text{Products.Brand\_ID}=\text{Brands.ID}} \rho_{\text{Brand\_Name/Name}}(\text{Brands})\Big)$$

Steps: (1) rename `Name` in Products to `Product_Name`; (2) rename `Name` in Brands to `Brand_Name` — this avoids an ambiguous attribute name; (3) join on the brand identifier; (4) project to only the three desired columns.

Left Semi Join $\ltimes$ and Right Semi Join $\rtimes$

A semi join tests for the *existence* of a match but does **not** append data from the other side. Only attributes from one relation appear in the result.

Left Semi Join $\ltimes$

Returns all tuples of the **left** relation for which at least one matching tuple exists in the right relation. Only the columns of the left relation are included.
Syntax: $X \ltimes_{X.A=Y.A} Y$

Right Semi Join $\rtimes$

Symmetric counterpart: returns tuples of the **right** relation that have a match in the left relation, using only the right relation's columns.

Examples

Use Case **"Which customers have placed at least one order?"**
Customers $\ltimes_{\text{Customers.ID=Orders.Customer\_ID}}$ Orders
Each customer appears at most once in the result (no row-per-order duplication), and no order data is added to the output.

Left Outer Join $\bowtie_L$ and Right Outer Join $\bowtie_R$

Outer joins **preserve** tuples even when no match exists in the other relation. Missing attribute values are filled with `NULL`.

Left Outer Join $\bowtie_L$

All tuples from the **left** relation are kept. Matching tuples from the right are appended; where no match exists the right-side columns are filled with `NULL`.

Right Outer Join $\bowtie_R$

All tuples from the **right** relation are kept. Matching tuples from the left are appended; `NULL`-filled otherwise.

Examples

Example — Left Outer Join $X \bowtie_L Y$: X has entries A_1, A_2, A_3; Y matches only A_1 and A_3.
Result: A_1 (with Y data), A_2 (right columns = `NULL`), A_3 (with Y data).

Use case: "Get all customers and their orders, including customers who haven't placed any order yet."

Full Outer Join $\bowtie_{LR}$ and Anti Join $\triangleright$

Full Outer Join $\bowtie_{LR}$

All tuples from **both** sides are preserved. `NULL`-filled for missing matches on either side.

$$R_1 \bowtie_{LR} R_2 \;=\; (R_1 \bowtie_L R_2) \cup (R_1 \bowtie_R R_2)$$

Example: $X \bowtie_{LR} Y$ where $X = \{A_1, A_2\}$ and $Y = \{A_1, A_3\}$ produces three result tuples: (A_1, match), $(A_2, \texttt{NULL})$, $(\texttt{NULL}, A_3)$.

Anti Join $\triangleright$

Returns tuples from R_1 for which there is **no** matching tuple in R_2 — the complement of a semi join.

$$X \;\triangleright_{X.A = Y.A}\; Y$$

Example: if $X = \{A_1, A_2\}$ and Y matches only A_1, the anti join returns only A_2.

Examples

Use Case **"Find customers who have *never* placed an order."**
Customers $\triangleright_{\text{ID}=\text{Customer\_ID}}$ Orders

Laws: Commutative and Associative

Query optimisers exploit algebraic laws to reorder operations without changing the result, thereby choosing the most efficient execution plan. The following laws hold for relational algebra:

Commutativity

- Union: $A \cup B = B \cup A$
- Intersection: $A \cap B = B \cap A$
- Cartesian Product: $A \times B = B \times A$
- Join: $A \bowtie B = B \bowtie A$

Associativity

- Union: $(A \cup B) \cup C = A \cup (B \cup C)$
- Intersection: $(A \cap B) \cap C = A \cap (B \cap C)$
- Cartesian Product: $(A \times B) \times C = A \times (B \times C)$
- Join: $(A \bowtie B) \bowtie C = A \bowtie (B \bowtie C)$

These laws allow a DBMS to, for example, join the *smallest* two tables first, drastically reducing the size of intermediate results.

Laws: Restriction Rules (!)

Note: Not Required for Exam

The following equivalences are important for understanding query optimisation but will not be examined.

Restrictions are cheap to apply and can often be *pushed down* closer to the base relations, reducing the number of tuples processed by subsequent (more expensive) operations.

Key Equivalences

- Chaining and merging: $\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_2}(\sigma_{C_1}(R)) = \sigma_{C_1 \wedge C_2}(R)$
- Distributing over union: $\sigma_C(R_1 \cup R_2) = \sigma_C(R_1) \cup \sigma_C(R_2)$
- Distributing over join: $\sigma_C(R_1 \bowtie R_2) = \sigma_C(R_1) \bowtie \sigma_C(R_2)$ (push restrictions *into* the join operands)
- Commuting with projection: $\sigma_C(\pi_A(R)) = \pi_A(\sigma_C(R))$ (apply restriction before projection)
- Distributing disjunction: $\sigma_{C_1}(R) \vee \sigma_{C_2}(R) = \sigma_{C_1}(R) \cup \sigma_{C_2}(R)$

Additional Laws (!)

Note: Not Required for Exam

These rules deepen understanding of the algebra but are not examinable.

Pushing Conditions to Where They Belong

If a restriction condition C only references attributes of R_2, it can be moved inside the join:

$$\sigma_{C_{\text{on } R_2}}(R_1 \bowtie R_2) \;=\; R_1 \bowtie \sigma_{C_{\text{on } R_2}}(R_2)$$

Applying the filter to R_2 *before* the join reduces the number of tuples that need to be combined.

Division as the Inverse of Cartesian Product

$$R_1 = (R_1 \times R_2) \div R_2$$

Just as multiplication and division are inverse arithmetic operations, division undoes a Cartesian product. This identity confirms the intuitive meaning of division.

These rules collectively form the foundation on which real-world DBMS query

Table of Contents

Why Normal Forms?

The Problem: Poorly Designed Relations

A naïvely designed table that stores "everything in one place" leads to a range of systematic problems collectively called **anomalies**.

- **Data Redundancy:** The same fact (e.g. an artist's debut year) is stored in every row that mentions that artist. This wastes storage and, more importantly, opens the door to inconsistency.
- **Update Anomalies:** If a fact is stored in n places and we update only $k < n$ of them, the database becomes contradictory.
- **Insertion Anomalies:** We cannot record a new fact without also supplying unrelated data. Example: we cannot add a new artist until that artist releases an album.
- **Deletion Anomalies:** Deleting a record accidentally removes other unrelated facts. Example: deleting the last album of an artist also deletes all information about that artist.
- **Complex Queries:** Denormalised tables often require intricate SQL (self-joins, aggregations, deduplication) just to produce correct results for simple questions.

The Solution: Normal Forms

What Are Normal Forms?

Normal forms (NF) are a series of progressive rules for organising relation schemas. Each rule eliminates a specific class of redundancy or dependency problem.

Goal

- Reduce redundancy and unnecessary attribute dependencies
- Eliminate update, insertion, and deletion anomalies
- Produce a clean, maintainable schema

Key Insight

Normal forms are *cumulative*: a relation in *n*NF must also satisfy all lower normal forms. There is no skipping

Hierarchy of Normal Forms

- Commonly used: 1NF, 2NF, 3NF, BCNF
- Less common: 4NF, 5NF
- Rarely used: 6NF, EKNF, ETNF, DKNF

Two Key Algorithms

- **Synthesis Algorithm** $\rightarrow$ 3NF (lossless *and* dependency-preserving)
- **Decomposition Algorithm** $\rightarrow$ BCNF

Functional Dependencies

Definition

A **functional dependency (FD)** $A \to B$ holds in a relation if, whenever two tuples agree on the value(s) of A, they must also agree on the value(s) of B. We say "A *functionally determines* B".

Notation

- $A \to B$ — "A determines B"
  Knowing A's value fixes B's value exactly.
- $AB \to C$ — "A and B together determine C"
  Both attributes on the left are required.
- $A \twoheadrightarrow B$ — "A multi-determines B"
  A determines a *set* of B values (multivalued dependency, covered in 4NF).

Examples

- `StudentID` $\to$ `StudentName`
  Each student ID maps to exactly one name.
- `CourseID` $\to$ `CourseName`
  Each course ID maps to exactly one name.
- `AlbumID` $\to$ `Title, MainArtist, Genre, Rele`
  The album ID determines all other album-level attributes.
- `AlbumID, Song` $\to$ `FeaturedArtist`

Types of Functional Dependencies

Understanding the *type* of an FD is essential for knowing which normal form it violates and how to fix it.

- **Trivial FD:** The dependent is a subset of the determinant. Example: $AB \to B$. Always true by definition; carries no information about the data.
- **Non-trivial FD:** The dependent is *not* part of the determinant. Example: $A \to B$ where $B \notin A$. These are the interesting dependencies to analyse.
- **Fully Functional FD:** $AB \to C$ holds, and *neither* $A \to C$ nor $B \to C$ alone holds. Both A and B are genuinely needed.
- **Partial FD:** $AB \to C$ holds, but also $A \to C$ alone holds. B is redundant in the determinant. Partial FDs are the target of 2NF.
- **Transitive FD:** $A \to B$ and $B \to C$ implies $A \to C$ transitively. The attribute C does not depend *directly* on A. Transitive FDs are the target of 3NF.
- **Multivalued FD:** $A \twoheadrightarrow B$ — knowing A determines an entire *set* of B values, independently of other attributes. Multivalued FDs are the target of 4NF.

Armstrong Axioms

Named after William W. Armstrong (1974). These inference rules allow us to *derive* all functional dependencies that logically follow from a given set F.

Original Three Axioms

1. **Reflexivity:**
   If $B \subseteq A$, then $A \rightarrow B$.
   A set always determines its own subsets.
2. **Augmentation:**
   If $A \rightarrow B$, then $AC \rightarrow BC$.
   Adding the same attribute(s) to both sides preserves the FD.
3. **Transitivity:**
   If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.

Derived Rules

4. **Union:**
   If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow BC$.
5. **Decomposition:**
   If $A \rightarrow BC$, then $A \rightarrow B$ and $A \rightarrow C$.
6. **Pseudotransitivity:**
   If $A \rightarrow B$ and $CB \rightarrow D$, then $CA \rightarrow D$.

Completeness and Soundness

The Armstrong axioms are both **complete** (every FD that holds can be derived) and **sound** (every derived FD actually holds). They form the foundation for all

Keys Revisited

Before studying normal forms in detail, we need precise definitions of *keys*.

- **Superkey:** A set $A \subseteq R$ such that $A \rightarrow R$ (i.e. A determines *all* attributes of R). May contain redundant attributes.
- **Candidate Key:** A *minimal* superkey — removing any single attribute makes it no longer a superkey.
  - Every candidate key is a superkey, but not vice versa.
  - A is a candidate key if $A \rightarrow R$ and no proper subset of A also determines R.
- **Primary Key:** The candidate key chosen by the database designer as the principal identifier.
- **Prime Attribute:** An attribute that is part of *at least one* candidate key.
- **Non-Prime Attribute:** An attribute that is *not* part of any candidate key. Non-prime attributes are the focus of most normalisation rules.

Hierarchy

Superkeys $\supset$ Candidate Keys $\supset$ Primary Key (exactly one)

First Normal Form (1NF)

Definition

A relation is in **1NF** if every attribute contains only **atomic** (indivisible) values, and every entry in a column is of the same data type. There are no repeating groups and no multi-valued attributes.

What Violates 1NF?

- A `Songs` column containing `"Would've...; Karma"` (two values in one cell).
- A column containing arrays, sets, or comma-separated lists.
- Repeating groups (e.g. `Phone1`, `Phone2`, `Phone3` columns).

How to Fix It

- Give each atomic value its own row.
- After the fix: one row per (Album, Song) pair.
- The primary key becomes composite: (`AlbumID`, `Song`).

Side Effect

Applying 1NF typically makes the table *longer* (more rows), but each row now represents exactly one atomic fact. Redundancy may increase temporarily — this is addressed by 2NF and 3NF.

1NF – Example

Before (NOT in 1NF):

AlbumID	Songs	Main Artist	...
A100	Song1; Song2	Taylor Swift	...

After (1NF — one song per row):

AlbumID	Song	Featured Artist	Main Artist	...
A100	Would've, Could've, Should've	—	Taylor Swift	...
A100	Karma	Ice Spice	Taylor Swift	...
A101	Snow on the Beach	Lana Del Rey	Taylor Swift	...

Still a Problem

Each row is now atomic — **1NF satisfied**. However, `Main Artist`, `Debut`, `Genre`, and `Release` repeat for every song of the same album. This redundancy is addressed by **2NF**.

Second Normal Form (2NF)

Definition

A relation is in **2NF** if it is in 1NF and every non-prime attribute is **fully functionally dependent** on the *entire* primary key (i.e. there are no partial dependencies).

- **Partial dependency:** A non-key attribute depends on only *part* of a composite primary key.
- After 1NF our primary key is `(AlbumID, Song)`. But `Title`, `MainArtist`, `Genre`, and `Release` all depend *only* on `AlbumID` (not on `Song`) — a partial dependency!
- **Fix:** Split into two tables:
  - `Albums(AlbumID, Title, MainArtistID, Genre, Release)`
  - `Songs(AlbumID, Song, FeaturedArtist)`
- Now every non-prime attribute depends on the *whole* key of its respective table.

Important Insight

2NF problems can *only* arise when the primary key is **composite**. If the PK is a single attribute, the relation is automatically in 2NF (assuming it is already in 1NF).

Third Normal Form (3NF)

Definition

A relation is in **3NF** if it is in 2NF and no non-prime attribute is **transitively dependent** on the primary key. Every non-prime attribute must depend *directly* on the PK, not through another non-prime attribute.

- **Transitive dependency:** $PK \to A \to B$, where A is non-prime. B does not depend directly on the PK; it reaches the PK via A.
- In our `Albums` table:
  `AlbumID` $\to$ `MainArtist` $\to$ `Debut`
  The artist's debut year depends on the artist, not on the album.
- **Fix:** Extract a separate `Artists` table:
  - `Albums(AlbumID, Title, ArtistID, Genre, Release)`
  - `Artists(ArtistID, Name, Debut)`
  - `Songs(AlbumID, Song, FeaturedArtist)`

Why 3NF Matters in Practice

Most production databases are designed to 3NF. It eliminates the most common anomalies with reasonable decomposition effort. The **Synthesis Algorithm** always produces a 3NF schema that is both lossless and dependency-preserving.

Boyce-Codd Normal Form (BCNF)

Definition

A relation is in **BCNF** (sometimes called *3.5NF*) if it is in 3NF and for *every* non-trivial FD $A \to B$, A is a **superkey**.

- BCNF is **strictly stronger** than 3NF: it catches edge cases where 3NF still permits anomalies.
- In 3NF, the right-hand side of an FD is allowed to be a prime attribute; BCNF *disallows* this exception.
- **Example violation:** `Releases(ID, Edition, LabelID, Label)`
  FDs: `LabelID` $\to$ `Label` and `Label` $\to$ `LabelID`.
  Neither `LabelID` nor `Label` is a superkey, yet they determine each other — a BCNF violation.
- **Fix:** Decompose into:
  `Releases(ID, Edition, LabelID)` and `Labels(LabelID, Label)`

Trade-off

BCNF *may not preserve all FDs* after decomposition. If dependency-preservation is critical, 3NF (via the Synthesis Algorithm) may be preferred.

BCNF – Example

Before (not in BCNF):

ID	Edition	LabelID	Label
A100	3am Edition	REP	Republic Records
A101	Late Night Edition	UMG	Universal Music

FDs: $\{\texttt{ID, Edition}\} \rightarrow \texttt{LabelID} \rightarrow \texttt{Label}$ and $\texttt{Label} \rightarrow \texttt{LabelID}$.
`LabelID` is **not** a superkey, yet $\texttt{LabelID} \rightarrow \texttt{Label}$ — **BCNF violation**!

After (BCNF):

Releases		
ID	**Edition**	**LabelID**
A100	3am Edition	REP
A101	Late Night Edition	UMG

Labels	
LabelID	**Label**
REP	Republic Records
UMG	Universal Music

Now every determinant (`ID`+`Edition` and `LabelID`) is a superkey. BCNF satisfied.

4NF (!)

Definition (!)

A relation is in **4NF** if it is in BCNF and has no **non-trivial multi-valued dependencies (MVDs)**.
An MVD $A \twoheadrightarrow B$ is *non-trivial* if $B \not\subseteq A$ and $A \cup B \neq R$.

- **The Problem:** `CustomerCode` $\twoheadrightarrow$ `Phones` and `CustomerCode` $\twoheadrightarrow$ `Addresses`.
  Phones and addresses are *independent* sets associated with the same customer.
- After 1NF, storing both in one table forces a *Cartesian product*: every phone is paired with every address, producing redundant and potentially spurious rows.
- **Fix:** Decompose into two tables:
  - `Customers_Phones(CustomerCode, Phone)`
  - `Customers_Addresses(CustomerCode, Address)`
- Now each independent multi-valued fact lives in its own table; no Cartesian product arises.

5NF (!)

Definition (!)

A relation is in **5NF** (also called *Project-Join Normal Form, PJNF*) if it is in 4NF and cannot be further decomposed in a **lossless** way.

- **Lossless decomposition:** Decomposing R into $R_1, R_2, \ldots$ is lossless if $R_1 \bowtie R_2 \bowtie \cdots = R$ exactly (no spurious tuples).
- If decomposing into 2 tables produces extra (spurious) tuples when joined back, the decomposition is *lossy* and a 3-table decomposition is required.
- **Example:** A `Lecturer-Course-Subject` table with complex business constraints (which lecturers teach which subjects in which courses) may need 3 separate tables to decompose losslessly.
- 5NF deals with **join dependencies** — generalisations of multi-valued dependencies.
- Rarely implemented in practice; most experts stop at 3NF or BCNF.

Practical Note

Verifying 5NF requires checking all possible join dependencies, which is computationally expensive and usually unnecessary for typical business applications.

Even Higher Normal Forms (!)

Beyond 5NF, a number of further normal forms have been defined, mostly in academic research.

- **6NF:** Every relation contains the primary key and *at most one* non-key attribute. Useful for **temporal databases** where attributes change independently over time. Proposed by C. J. Date, Hugh Darwen, and Nikos Lorentzos (2002).
- **EKNF (Elementary Key NF):** A stricter version of 3NF that rules out certain 3NF-compliant but anomalous schemas.
- **ETNF (Essential Tuple NF):** A stricter version of 4NF; eliminates certain redundancies not addressed by 4NF.
- **DKNF (Domain-Key NF):** Every integrity constraint in the relation is a *logical consequence* of domain constraints and key constraints. Considered the "ultimate" normal form — if achieved, all anomalies are guaranteed to be absent.

Practical Guidance

Most of these forms are **academic**. In practice:

- Aim for **3NF minimum** for all production schemas.
- **BCNF is ideal** when dependency preservation can be verified externally.

Normal Forms: The Big Picture (!)

Constraint	UNF	1NF	2NF	3NF	BCNF	4NF	5N
Unique rows		✓	✓	✓	✓	✓	✓
Atomic values (1NF)			✓	✓	✓	✓	✓
No partial FDs (2NF)				✓	✓	✓	✓
No transitive FDs (3NF)					✓	✓	✓
Every FD has superkey determinant						✓	✓
No nontrivial MVDs (4NF)							✓
No lossy decompositions (5NF)							
Only trivial join dependencies (6NF)							

Reading This Table

Each column represents a normal form. A ✓ means the constraint is *guaranteed* by that normal form and all higher ones. The table shows how each NF strictly extends its predecessor.

Algorithms for Normal Forms

Why Use Algorithms?

Manually decomposing large relations is error-prone and may miss violations or produce redundant tables. Two systematic algorithms provide guarantees.

Synthesis Algorithm → 3NF

- Produces a **lossless** decomposition
- Also **dependency-preserving** (all original FDs are representable in the result)
- Preferred when preserving all FDs matters

Decomposition Algorithm → BCNF

- Produces a **lossless** decomposition
- **May not preserve** all functional dependencies
- Preferred when the schema must be in BCNF and FD loss is acceptable

Prerequisite: Canonical Cover

Both algorithms require the **Canonical Cover** F_c as input — a minimal, equivalent, redundancy-free representation of the FD set F.

Examples

Canonical Cover

Definition

The **canonical cover** F_c of a set of FDs F is a **minimal equivalent** set of FDs: it preserves exactly the same dependencies as F but contains no redundancy on either side of any FD.

Four Steps to Compute F_c:

1. **Reduce Left Sides:** For each FD $AB \to C$, check if $A \to C$ or $B \to C$ alone. If yes, remove the redundant attribute from the left side. Repeat until no further reduction is possible.
2. **Reduce Right Sides:** For each FD $A \to BC$, check whether $A \to C$ can be derived without using $A \to C$ directly (i.e. using the remaining FDs). If yes, remove C from the right side.
3. **Delete Empty FDs:** Remove any FD of the form $A \to \emptyset$ (arises when the right side was fully eliminated in step 2).
4. **Join FDs with Identical Left Side:** Merge $A \to B$ and $A \to C$ into $A \to BC$.

The order of steps 1 and 2 may vary by textbook, but the result is equivalent.

Canonical Cover – Example

Variables: A = City, B = Country, C = Full Station Name, D = State, E = Train Station

Initial FDs: $ABE \to C$, $E \to AC$, $E \to A$, $A \to BD$ **Candidate key:** E

Step 1 — Reduce Left Sides:
We have $A \to BD$ and $E \to A$, so $E \to B$ transitively.
Therefore B is redundant in $ABE \to C$: reduce to $E \to C$.
Result: $\{E \to C,\ E \to AC,\ E \to A,\ A \to BD\}$

Step 2 — Reduce Right Sides:
In $E \to AC$: since $E \to A$ already exists, A is redundant on the right. Reduce $E \to AC$ to $E \to C$. But $E \to C$ already exists, so $E \to AC$ becomes $E \to \emptyset$ (after also removing C).
Result: $\{E \to \emptyset,\ E \to C,\ E \to A,\ A \to BD\}$

Step 3 — Delete Empty FDs:
Remove $E \to \emptyset$.
Result: $\{E \to C,\ E \to A,\ A \to BD\}$

Step 4 — Join FDs with Same Left Side:
Merge $E \to C$ and $E \to A$ into $E \to AC$.

Synthesis Algorithm ($\rightarrow$ 3NF)

Input

Relation R, canonical cover F_c, candidate keys of R.

Four Steps:

1. **Calculate** the canonical cover F_c (see previous slide).
2. **Create one relation per FD in F_c:** For each $A \rightarrow B \in F_c$, define $R_i = A \cup B$.
3. **Ensure a candidate key is covered:** If none of the relations from step 2 contains a candidate key of R, add a new relation $R_k =$ (attributes of one candidate key).
4. **Remove redundant relations:** Delete any R_i that is entirely contained in another R_j ($R_i \subseteq R_j$).

Examples

Example FDs: $A \rightarrow EBD$, $F \rightarrow CDB$, $CD \rightarrow EF$ Candidate key: AF

Step 2: $R_1 = \{A, B, D, E\}$, $R_2 = \{B, C, D, F\}$, $R_3 = \{C, D, E, F\}$

Step 3: None of R_1–R_3 contains AF, so add $R_4 = \{A, F\}$.

Step 4: No relation is a subset of another.

Decomposition Algorithm ($\rightarrow$ BCNF)

Input

Relation R, functional dependencies F.

Three Steps:

1. **Initialise:** $Z = \{R\}$.
2. **Decompose BCNF violators:** While any $R_i \in Z$ contains an FD $A \rightarrow B$ where A is *not* a superkey of R_i:
   - Split R_i into $R_{i1} = A \cup B$ and $R_{i2} = R_i \setminus B$.
   - Replace R_i in Z with R_{i1} and R_{i2}.
3. **Repeat** until all relations in Z are in BCNF.

Examples

Example $R = \{A, B, C, D, E\}$; FDs: $ABE \rightarrow C$, $E \rightarrow AC$, $E \rightarrow A$, $A \rightarrow BD$
Violation: $A \rightarrow BD$ (A is not a superkey of R).
Decompose: $R_1 = A \cup BD = \{A, B, D\}$, $\quad R_2 = ABCDE \setminus BD = \{A, C, E\}$
$Z = \{R_1\{ABD\},\ R_2\{ACE\}\}$
Check R_1: candidate key A; $A \rightarrow BD$ — BCNF satisfied.
Check R_2: candidate key E; $E \rightarrow AC$ — BCNF satisfied.
Result: $R_1\{ABD\},\ R_2\{ACE\}$

Table of Contents

The Big Picture

Database Design Is an Iterative Process

From a vague idea to a running database, every project passes through the same chain of refinements:

Requirements → User Story Map / Feature Spec → Concept Draft → **ERM** → Implementation Draft → **Relational Schema** → Physical Design → **Database Implementation**

Key Principles

- Each step **refines** the previous one – no step is skipped in a professional project
- Mistakes made early (in requirements) are the **most expensive** to fix later
- A well-designed schema prevents the majority of data integrity problems and performance issues before a single line of application code is written

Examples

Recommended Design Tools **diagrams.net** (free, browser-based, formerly draw.io)
| **MySQL Workbench** | **Lucidchart**

Requirements Engineering (!)

Why Requirements Engineering Is Critical

60 % of all errors in software projects originate during requirements engineering.

The cost to fix a defect grows dramatically the later it is discovered:

- Caught during *requirements gathering*: baseline cost $= 1\times$
- Caught during *implementation*: $\approx 20\times$ more expensive
- Caught during *production rollout*: $\approx 100\times$ more expensive

Four Phases of Requirements Engineering

1. **Identification** – gather requirements through interviews, surveys, focus groups, observation, and prototyping; involve all stakeholders early
2. **Documentation** – categorise as functional / non-functional / quality / constraints; use models (ERM, User Story Maps); prioritise ruthlessly
3. **Validation** – verify each requirement after implementation; monitor continuously; detect contradictions before they become bugs
4. **Management** – communicate changes to all stakeholders; create, revise, and retire requirements in a controlled way; document every change decision

Extracting Entities from Requirements

Natural Language → ERM Elements

Real requirements arrive as text. A systematic reading strategy maps language constructs to ERM components.

Examples

Example Requirement Text

- *"Every car has an owner."*
- *"Every person can have multiple cars."*
- *"A car has a licence plate number, which is registered at one registration authority."*

Reading Strategy

- **Nouns** → candidates for **entities** or **attributes**:
  car, *owner / person*, *licence plate number*, *registration authority*
- **Verbs / prepositions** → candidates for **relationships**:
  "*has*" (person → car) "*registered at*" (licence plate → registration authority)

Entity-Relationship Model (ERM)

Background

The ERM was introduced in **1976 by Peter Chen** (Chen's notation). It is a **conceptual model** for representing data requirements visually, *before* any implementation decision is made. It is DBMS-agnostic and does not prescribe a normalisation form.

Core Components

Entity	An identifiable, distinguishable real-world object (Person, Product, Order)
Attribute	A descriptive property of an entity (Name, Price, Date)
Relationship	A meaningful link between two or more entities (Person DRIVES Car)
Weak Entity	Exists only in relation to a *strong* entity and cannot be uniquely identified without it (e.g. Contract of an Employee)
Subtype	A specialisation of a supertype that inherits its at-

ERM Notation (Chen's)

Shape Dictionary

- **Rectangle** $\Rightarrow$ Entity
- **Diamond** $\Rightarrow$ Relationship
- **Ellipse** $\Rightarrow$ Attribute
- **Double rectangle** $\Rightarrow$ Weak entity
- **Double diamond** $\Rightarrow$ Identifying relationship (connects weak entity to its owner)
- **Underlined text** $\Rightarrow$ Primary key attribute
- **Double ellipse** $\Rightarrow$ Multivalued attribute (e.g. a person can have multiple phone numbers)
- **Dashed ellipse** $\Rightarrow$ Derived attribute (computed from other attributes, e.g. Age from Birthday)

Reading a Chen Diagram

Entities are connected by labelled relationship diamonds. Cardinality annotations (1, n, m) are placed on the connecting lines between the relationship diamond and

Cardinalities in Chen's Notation

Definition

A **cardinality** defines how many instances of one entity can be associated with how many instances of another entity through a given relationship.

The Three Basic Cardinalities

Notation	Meaning
1:1	Each instance of A is related to at most one instance of B, and vice versa
1:n	Each instance of A can be related to many instances of B; each instance of B is related to at most one A
m:n	Each instance of A can be related to many instances of B, and each instance of B can be related to many instances of A

Examples

Modified Chen's Notation (1 = 1, c = can, m = must)

Extended Cardinality Symbols

The basic 1:1 / 1:n / m:n notation does not capture optional participation. The **modified notation** adds two symbols:

c = "can" (0 or more / 0 or 1) – participation is *optional*
m = "must" (at least 1) – participation is *mandatory*

Symbol Table

Symbol	Meaning
1:1	exactly 1 to exactly 1
1:c	exactly 1 to 0 or 1
1:m	exactly 1 to at least 1
1:mc	exactly 1 to 0 or more
c:c	0 or 1 to 0 or 1 ($\rightarrow$ maps to basic 1:1)
c:mc	0 or 1 to 0 or more ($\rightarrow$ maps to basic 1:n)
mc:mc	0 or more to 0 or more ($\rightarrow$ maps to basic m:n)

Crow's Foot Notation

What Is Crow's Foot?

Crow's Foot notation is an alternative ERM notation widely used in professional database tools such as **MySQL Workbench**, **ERwin**, and **draw.io**. It is more compact than Chen's notation for large, complex schemas.

Symbol Vocabulary

- $|$ exactly one (mandatory, singular)
- $||$ one and only one
- $\circ$ zero or one (optional)
- $<$ (crow's foot) many

Symbols are always placed at the end of the line *closest to the entity they modify*.

Common Combinations

- $|<$ one-to-many (mandatory on one side)
- $\circ<$ zero-or-one-to-many
- $\circ|$ $|<$ zero-or-one to one-or-many

Extended ERM (EERM)

What Does the EERM Add?

The **Extended Entity-Relationship Model** enriches the original ERM with object-oriented concepts, primarily to model **inheritance hierarchies** and **self-referencing structures**.

Core Concepts

- **Generalisation** (bottom-up): abstract a common supertype from multiple specific subtypes.
  Example: Employee and Customer → generalised to Person
- **Specialisation** (top-down): derive specific subtypes from a general supertype.
  Example: Person → specialised into FullTimeEmployee and PartTimeEmployee
- **Attribute redefinition (!):** a subtype may redefine (override) inherited attributes with a more specific type or constraint
- **Recursive relationships (!):** an entity relates to itself.
  Example: Employee "manages" Employee (manager-subordinate hierarchy)

Exercise: ERM

Task

Transfer the following verbal descriptions into a complete **ERM diagram** using Chen's (or Crow's Foot) notation. Add realistic attributes to all entities. Annotate cardinalities for every relationship.

Exercise Levels

- **Simple:** Exercise 6A
- **Intermediate:** Exercise 6B
- **Advanced:** Exercise 6C

Examples

Tool & Time Use **diagrams.net** (`app.diagrams.net`) as your diagramming tool – it is free, runs in the browser, and exports to PDF or XML.

Time: 30 minutes.

Exercise: EERM

Task

Transfer the following verbal descriptions into a complete **EERM diagram**. Your model must include at least one **generalisation or specialisation** hierarchy. Annotate all cardinalities and mark inherited attributes.

Exercise Levels

- **Simple:** Exercise 7A
- **Intermediate:** Exercise 7B
- **Advanced:** Exercise 7C

Examples

Tool & Time Use **diagrams.net** (`app.diagrams.net`).

Time: 30 minutes.

Implementation Draft – Rules

From ERM to Relational Schema

Converting an ERM / EERM into a set of relations follows a fixed set of mapping rules. Apply them in order:

Mapping Rules

1. Every **strong entity** becomes its own relation with a **primary key**
2. Every **weak entity** becomes its own relation; its PK is a **composite key** that includes the PK of its owning (strong) entity
3. Every **many-to-many (m:n) relationship** becomes its own **junction relation** containing both foreign keys (and any relationship attributes)
4. **One-to-many (1:n)** relationships: add a **foreign key on the "many" side**; no separate relation is needed
5. **One-to-one (1:1)** relationships: add a foreign key on *either* side (prefer the optional participant); no separate relation is needed
6. **Multivalued attributes** become their own relation with a foreign key back to the originating entity

Implementation Draft – Example

Starting ERM

Author(*Name, Birthday*) `-[writes]->` **Book**(*Barcode, PageCount, ChapterID*)

Cardinality of *writes*: **m:n** (one author can write many books; one book can have many authors)

Resulting Relational Schema

- **Author**(AuthorID, Name, Birthday)
- **Book**(Barcode, PageCount, ChapterID)
- **Author_Book**(AuthorID, Barcode) – *junction table for the m:n relationship*

Examples

Rules Applied

- Rule 1: Author and Book are strong entities → each gets its own relation with a surrogate PK

From Schema to SQL

The Final Step: Physical Implementation

Once the implementation draft (relational schema) is complete, **SQL DDL statements** are written to create the physical database objects.

DDL Elements Used

- `CREATE TABLE` – one statement per relation in the schema
- `PRIMARY KEY` – enforces entity integrity; each row is uniquely identifiable
- `FOREIGN KEY ...REFERENCES` – enforces referential integrity between tables
- `NOT NULL` – ensures mandatory attributes are always populated
- `UNIQUE` – enforces candidate key constraints beyond the PK
- `CHECK` – enforces domain constraints (e.g. `age > 0`)

Coming Up

SQL – both DDL (schema creation) and DML (data manipulation: `SELECT`, `INSERT`, `UPDATE`, `DELETE`) – is covered in detail in the next section.

Exercise: EERM & Relational Schema

Task

Transfer the verbal descriptions into a complete **EERM diagram** *and* derive the corresponding **relational schema** from it. Apply all mapping rules systematically.

Exercise Levels

- **Simple:** Exercise 8A
- **Intermediate:** Exercise 8B
- **Advanced:** Exercise 8C

Deliverables

1. EERM diagram in **diagrams.net**
2. Written relational schema using standard notation:
   `RelationName(PK, attr1, attr2, FK)`

Examples

Time **40 minutes.**

Table of Contents

XAMPP – Overview

What Is XAMPP?

XAMPP is a free, open-source software bundle that installs a fully functional local web server and database environment in minutes – no manual configuration required.

- Developed by **Apache Friends** since **2002**
- Available for **Windows**, **macOS**, and **Linux** (hence "cross-platform")
- Used in this course to host a local **MySQL / MariaDB** database for all exercises

The Acronym Explained

X	Cross-platform
A	Apache HTTP Server
M	MariaDB (originally MySQL)
P	PHP
P	Perl

Why XAMPP?

Installing XAMPP

Download & Install

1. Download the installer from **apachefriends.org** (choose the version matching your OS)
2. Run the installer – *accepting the default paths is strongly recommended*
3. Default installation directories:
   - Windows: `C:\xampp`
   - macOS: `/Applications/XAMPP`

After Installation: The Control Panel

The **XAMPP Control Panel** is your central dashboard. It lists all bundled services:

- Apache, **MySQL**, FileZilla, Mercury, Tomcat

To start the database: click **"Start"** next to *MySQL*.
The status indicator turns **green** when the service is running successfully.

Examples

Typical First-Time Pitfall On Windows, port conflicts with Skype or IIS are

Starting and Accessing XAMPP

Starting the Database Service

- Open the XAMPP Control Panel and click **Start** next to *MySQL*
- Default port: **3306** (the MySQL/MariaDB standard port)
- Default credentials:
  - Username: `root`
  - Password: *(empty by default)*

Security Warning

An empty root password is acceptable *only* on a local development machine with no external network access. **Never** deploy XAMPP with default credentials in a production or publicly accessible environment!

Access Options

- **phpMyAdmin** (browser-based): navigate to `http://localhost/phpmyadmin`
- **MySQL command line**: available directly from the XAMPP shell
- **External SQL clients**: MySQL Workbench, DBeaver, DataGrip – connect to

phpMyAdmin

What Is phpMyAdmin?

phpMyAdmin is a free, browser-based administration tool for MySQL and MariaDB databases. It ships pre-installed with XAMPP and requires no additional setup.

Access it at: `http://localhost/phpmyadmin` (after starting MySQL in XAMPP)

Core Capabilities

- Create, modify, and drop **databases** and **tables**
- Execute **SQL queries** directly in the browser
- **Import** (CSV, SQL dump) and **export** data in multiple formats
- Manage **users and permissions**
- Browse and edit table contents visually (without writing SQL)

UI Layout

- **Left panel**: collapsible tree of databases and their tables
- **Right panel**: operations area – SQL editor, table structure view,

MySQL Workbench

What Is MySQL Workbench?

MySQL Workbench is the official, free GUI tool from Oracle for MySQL databases.
Download: `dev.mysql.com/downloads/workbench/`

It combines a SQL editor, visual schema designer, and server administration panel in one application.

Key Features

- **Visual schema design**: drag-and-drop ERM / schema diagrams
- **SQL editor**: auto-completion, syntax highlighting, query history
- **Server administration**: user management, performance monitoring, log viewer
- **Data migration wizard**: migrate data from other DBMS platforms
- **Forward engineering**: generate `CREATE` SQL scripts directly from your ERM diagram
- **Reverse engineering**: auto-generate an ERM diagram from an existing live database

DBeaver – Universal Database Client

What Is DBeaver?

DBeaver is a free, open-source, cross-platform database client that supports virtually every database system through JDBC drivers: MySQL, MariaDB, PostgreSQL, SQLite, Oracle, SQL Server, MongoDB, and many more.

Download: `dbeaver.io` (Community Edition is fully free)

Key Features

- **Universal connectivity**: one tool for all databases – no need to switch between MySQL Workbench, pgAdmin, and others
- **SQL editor**: syntax highlighting, auto-completion, query history, script execution with results shown inline
- **ER diagram viewer**: automatically reverse-engineers a schema into an entity-relationship diagram
- **Data editor**: browse and edit table contents visually, with filtering and sorting
- **Data export/import**: CSV, JSON, SQL dump, Excel – all built in
- **Query profiling**: built-in `EXPLAIN` plan viewer with a graphical

DBeaver – Connecting to MySQL

Setting Up a Connection

1. Open DBeaver and click **"New Database Connection"** (plug icon)
2. Select **MySQL** from the driver list and click Next
3. Enter the connection details:
   - **Host**: `localhost`
   - **Port**: `3306`
   - **Username**: `root`
   - **Password**: *(empty for XAMPP default)*
4. Click **Test Connection** – DBeaver downloads the JDBC driver automatically if not already installed
5. Click **Finish**

Examples

Why DBeaver over phpMyAdmin? DBeaver is a *native desktop application* – it is faster, supports keyboard shortcuts, handles large result sets better, and works offline without a running Apache web server. For writing and testing SQL queries, most developers prefer it over browser-based tools.

Docker – Running MySQL Without XAMPP

What Is Docker?

Docker packages an application and all its dependencies into a self-contained unit called a **container**. A container runs identically on any machine – no "it works on my laptop" problems.

For databases, Docker means: spin up a MySQL server in *one command*, destroy it just as easily, and run multiple versions side by side.

Starting a MySQL container

```
# Pull the official MySQL image and start a container
docker run --name my-mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=supermarket \
  -p 3306:3306 \
  -d mysql:8.0

# Connect with the MySQL CLI inside the container
docker exec -it my-mysql mysql -u root -psecret

# Stop and remove the container when done
```

Docker Compose – Multi-Container Setup

Real projects often need both a database *and* an application server. **Docker Compose** describes the entire stack in a single YAML file.

docker-compose.yml example

```
version: "3.9"
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: supermarket
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql   # persist data across
          restarts

  adminer:
    image: adminer                # lightweight phpMyAdmin
        alternative
    ports:
```

Tool Comparison

Tool	Type	DB Support	Best For
XAMPP	Local server bundle	MySQL/MariaDB	Quick local setup
phpMyAdmin	Browser GUI	MySQL/MariaDB	Simple admin tasks
MySQL Workbench	Desktop GUI	MySQL only	ERM design, DDL
DBeaver	Desktop GUI	Any (via JDBC)	Daily SQL work
Docker	Container runtime	Any image	Reproducible envs

Recommendation for this course

- **Database server**: XAMPP (Windows/macOS) or Docker (any OS)
- **Schema design**: MySQL Workbench (ERM forward-engineering)
- **Daily SQL queries**: DBeaver

Table of Contents

What is SQL?

SQL stands for *Structured Query Language*. Its original name was *SEQUEL* (Structured English Query Language), developed in **1974 by IBM Research** (Donald Chamberlin and Raymond Boyce) as a language for interacting with their System R relational prototype.

Standardisation History

- **SQL-86** – first ANSI/ISO standard
- **SQL-92** – major expansion; still the baseline for most textbooks
- **SQL:1999** – added recursion, triggers, object extensions
- **SQL:2003** – window functions, XML support
- **SQL:2016** – JSON support, row pattern matching

Most relational DBMSs (MySQL, PostgreSQL, Oracle, SQL Server) implement the core standard but each adds its own **dialect** – small differences in syntax, functions, and extensions.

Case Sensitivity

SQL keywords (`SELECT`, `FROM`, ...) are **not** case-sensitive. String *values* however **are** case-sensitive: `'Hamburg'` $\neq$ `'hamburg'`.

SELECT – The Foundation

The SELECT statement retrieves data from one or more tables. It is the most frequently used SQL statement.

Full Syntax (clauses in writing order)

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s)
LIMIT number
```

- SELECT and FROM are the only **mandatory** clauses; all others are optional.
- GROUP BY is used together with aggregate functions; HAVING filters *groups* (like WHERE but for groups, not individual rows).

Execution Order (not writing order!)

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Understanding this order is essential: it explains, for example, why column aliases

SELECT – Basic Usage

Examples

```sql
-- Select all columns from a table
SELECT * FROM brand;

-- Select specific columns
SELECT name, address FROM brand;

-- Qualify column names (needed with multiple tables)
SELECT brand.name, brand.city FROM brand;
```

- Using * returns **all columns** – convenient for exploration but avoid in production code: column order can change when the table is altered, and transferring unneeded columns wastes network bandwidth and memory.
- Qualifying with `table.column` notation prevents **ambiguity** when joining multiple tables that share column names (e.g. both have an `id` column).

Examples

Tip: Explore first, then be specific When learning a new schema, start with

SELECT DISTINCT

Returning Only Unique Values

```sql
-- Return only unique values
SELECT DISTINCT country FROM brand;

-- Multiple columns: the combination must be unique
SELECT DISTINCT country, legal_form FROM brand;
```

- Without `DISTINCT`, SQL returns **all rows including duplicates**. This differs from pure relational algebra, which always produces sets (no duplicates).
- `DISTINCT` applies to the **entire row**, not just one column: in the second example, the pair `(country, legal_form)` must be unique.

Performance Warning

`DISTINCT` forces the database to sort or hash the entire result set to eliminate duplicates. On large tables this can be very expensive. Avoid it if your query is already guaranteed to return unique rows, or if you simply want to count (use `COUNT(DISTINCT column)` instead).

SELECT with LIMIT / TOP

Restricting the Number of Returned Rows

```sql
-- MySQL / MariaDB / PostgreSQL:
SELECT * FROM product LIMIT 10;

-- Skip first 20 rows, then return the next 10
SELECT * FROM product LIMIT 10 OFFSET 20;

-- SQL Server syntax:
SELECT TOP 10 * FROM product;
```

- `LIMIT` is essential when exploring large tables – without it a query on a table with millions of rows could transfer gigabytes of data.
- `OFFSET` enables **pagination**: to show page 3 with 10 rows per page, use `LIMIT 10 OFFSET 20`.

Always combine LIMIT with ORDER BY

Without an explicit `ORDER BY` the row order is **undefined** – the database may return a different set of rows each time. `LIMIT` without `ORDER BY` is only useful for a quick look at the data, never for reproducible results.

Aliases (AS)

Renaming Columns, Tables, and Expressions

```sql
-- Column alias
SELECT best_before_date AS bbd FROM stock;

-- Table alias (useful in JOINs)
SELECT p.name, p.price FROM product AS p;

-- Expression alias
SELECT AVG(price) AS avg_price FROM product;
```

- Aliases are **temporary**: they exist only for the duration of the query and do not change the underlying table.
- Table aliases reduce typing and dramatically improve readability in complex queries with multiple joins.
- Column aliases can be used in `ORDER BY` (MySQL allows this as an extension) but **not** in `WHERE` or `HAVING` – because those clauses are evaluated before `SELECT` assigns the alias.

Examples

Aggregate Functions

Aggregate functions perform a calculation on a **set of rows** and return a **single value**. They are the building block of statistical queries.

Common Aggregate Functions

```
SELECT COUNT(*) FROM product;          -- count all rows
SELECT COUNT(price) FROM product;      -- count non-NULL
    values
SELECT MIN(price) FROM product;
SELECT MAX(price) FROM product;
SELECT SUM(price) FROM product;
SELECT AVG(price) AS average_price FROM product;
```

- `COUNT(*)` counts **all rows** (including those with `NULL`); `COUNT(column)` counts only rows where the column is **not** `NULL`.
- Aggregate functions are most powerful when combined with `GROUP BY` to compute per-group statistics.

Cannot Use Aggregates in WHERE

Aggregate functions are computed *after* `WHERE` filtering. Use `HAVING` to filter on

WHERE – Filtering Rows

Basic Filtering

```
SELECT * FROM product WHERE price > 5;
SELECT name FROM brand WHERE city = 'Hamburg';
```

- `WHERE` filters **which rows** are included in the result – only rows for which the condition evaluates to `TRUE` are kept.
- String comparisons require **single quotes** (`'Hamburg'`), not double quotes.
- `NULL` comparisons are a common pitfall: `= NULL` always evaluates to `NULL` (unknown), never `TRUE`. Always use `IS NULL` or `IS NOT NULL`.
- Logical operators (`AND`, `OR`, `NOT`) can combine multiple conditions; use parentheses for grouping.

Logical Operator Precedence

`()` $\rightarrow$ `NOT` $\rightarrow$ `AND` $\rightarrow$ `OR`

When in doubt, add parentheses – they cost nothing and prevent subtle bugs caused by unexpected precedence.

WHERE – Logical Operators

AND, OR, NOT

```sql
-- AND: both conditions must be true
SELECT * FROM brand
WHERE city = 'Hamburg' AND legal_form = 'AG';

-- OR: at least one condition must be true
SELECT * FROM employee
WHERE department_id = 1 OR first_name = 'Mathis';

-- NOT: negates the condition
SELECT * FROM product WHERE NOT unit_id = 'KG';

-- Combining with parentheses
SELECT * FROM brand
WHERE legal_form = 'KG'
  AND (city = 'Hamburg' OR city = 'Berlin');
```

Common Mistake: Missing Parentheses

WHERE – Comparison and BETWEEN

Standard Operators and BETWEEN / IN

```sql
-- Standard comparison operators
SELECT * FROM product WHERE price >= 5;
SELECT * FROM product WHERE alcohol <> 0; -- != also works

-- BETWEEN (inclusive on both ends)
SELECT * FROM product WHERE price BETWEEN 2 AND 5;

-- IN (equivalent to multiple OR conditions)
SELECT * FROM brand
WHERE city IN ('Hamburg', 'Berlin', 'Muenchen');
```

- <> and != both mean "not equal"; <> is the SQL standard, != is widely supported as an alias.
- `BETWEEN` is **inclusive** on both bounds: `BETWEEN 2 AND 5` includes the values 2 and 5.
- `IN` is more readable than chaining many `OR` conditions and the query optimiser treats them equivalently. `NOT IN` is the logical negation.

WHERE – LIKE for Pattern Matching

Wildcard Characters

```sql
-- % matches zero, one, or many characters
SELECT * FROM employee WHERE name LIKE 'M%';      -- starts with M
SELECT * FROM brand    WHERE name LIKE '%GmbH';   -- ends with GmbH
SELECT * FROM product WHERE name LIKE '%milk%';   -- contains 'milk'

-- _ matches exactly one character
SELECT * FROM brand WHERE name LIKE '_ike';  -- 4-letter names ending 'ike'
SELECT * FROM brand WHERE name LIKE '%a_';  -- 2nd-to-last letter is 'a'
```

- % is the most commonly used wildcard; _ (underscore) is used when the exact position matters.
- In MySQL with the default collation, `LIKE` is **case-insensitive**. This behaviour depends on the table's collation setting.

WHERE – NULL Handling

Testing for NULL

```sql
-- Correct: use IS NULL / IS NOT NULL
SELECT * FROM stock WHERE storage_id IS NOT NULL;
SELECT * FROM stock WHERE best_before_date IS NULL;
```

= NULL Never Works

`WHERE column = NULL` always evaluates to `NULL` (unknown), **never** `TRUE`. The row is therefore never returned. This is one of the most frequent beginner mistakes in SQL.

NULL Semantics

- `NULL` means *unknown* or *not applicable* – it is **not** the same as zero, the empty string, or `false`.
- **NULL propagates**: any arithmetic or comparison involving `NULL` yields `NULL`. E.g. `NULL + 5 = NULL`, `NULL > 0 = NULL`.
- Design decision: use `NULL` for truly optional attributes where no value is

ORDER BY

Sorting Query Results

```sql
-- Ascending (default)
SELECT * FROM product ORDER BY price;
SELECT * FROM product ORDER BY price ASC;

-- Descending
SELECT * FROM product ORDER BY price DESC;

-- Multiple columns: primary then secondary sort
SELECT * FROM product ORDER BY brand_id, price DESC;
```

- Without `ORDER BY`, the row order is **undefined** – the database is free to return rows in any order, and this can vary between executions.
- Multiple columns: rows are first sorted by the first column; ties are broken by the second column, and so on. Each column can independently be `ASC` or `DESC`.
- `ORDER BY` is evaluated **after** `WHERE` and `GROUP BY` in the execution order.
- `NULL` values sort to the *end* in MySQL by default (`ASC`); other DBMSs (e.g. PostgreSQL) sort `NULL` first by default.

GROUP BY and HAVING

Grouping and Filtering Groups

```sql
-- Average price per brand
SELECT brand_id, AVG(price) AS avg_price
FROM product
GROUP BY brand_id;

-- Only brands with more than 10 products
SELECT brand_id, AVG(price), COUNT(*) AS product_count
FROM product
GROUP BY brand_id
HAVING COUNT(*) > 10;

-- Combining WHERE and HAVING
SELECT brand_id, AVG(price)
FROM product
WHERE price > 0
GROUP BY brand_id
HAVING AVG(price) > 5;
```

CASE WHEN (!)

Conditional Expressions in SELECT

```
SELECT name, price,
  CASE
    WHEN price > 10 THEN 'Expensive'
    WHEN price >= 5 THEN 'Medium'
    ELSE 'Cheap'
  END AS price_category
FROM product;
```

- `CASE WHEN` acts like an **if-else expression** inside a query – it returns different values depending on which condition is met.
- Conditions are evaluated **in order**; the first matching `WHEN` branch is returned and the rest are skipped.
- If no condition matches and there is no `ELSE`, `NULL` is returned.
- `CASE WHEN` can appear in `SELECT`, `ORDER BY`, and even inside aggregate functions (e.g. `SUM(CASE WHEN ...THEN 1 ELSE 0 END)`).

Examples

Subqueries (!)

A **subquery** (or inner query) is a SELECT statement nested inside another SQL statement.

Three Common Positions

```sql
-- 1. Subquery in FROM (derived table / inline view)
SELECT name
FROM (SELECT * FROM product WHERE price > 5) AS expensive;

-- 2. Subquery in WHERE with IN
SELECT name FROM department
WHERE id IN (
  SELECT department_id FROM product
  GROUP BY department_id
  HAVING COUNT(*) > 5
);

-- 3. Correlated subquery (references outer query row by row
    )
SELECT name,
  (SELECT AVG(p2.price) FROM product p2
   WHERE p2.brand_id = p.brand_id) AS brand_avg
```

EXISTS, ANY, ALL (!)

Set-Based Comparison Operators

```sql
-- EXISTS: true if subquery returns at least one row
SELECT * FROM employee AS e
WHERE EXISTS (
  SELECT 1 FROM product AS p
  WHERE p.department_id = e.department_id
);

-- ANY: true if any subquery value satisfies the condition
SELECT * FROM product
WHERE price > ANY (
  SELECT price FROM product WHERE department_id = 8
);

-- ALL: true if ALL subquery values satisfy the condition
SELECT * FROM product
WHERE price > ALL (
  SELECT price FROM product WHERE department_id = 3
);
```

Relational Algebra to SQL – Unary Operators

The table below maps each unary relational algebra operator to its SQL equivalent. Understanding this mapping helps you translate between the formal theory and practical query writing.

RA Operation	**RA Syntax**	**SQL Equivalent**
Restriction	$\sigma_C(R)$	`SELECT * FROM R WHERE C`
Projection	$\pi_A(R)$	`SELECT A FROM R`
Rename (table)	$\rho_N(R)$	`... FROM R AS N`
Deduplication	$\delta(\pi_A(R))$	`SELECT DISTINCT A FROM R`
Aggregation	$\gamma_{A,\,\mathrm{SUM}(B)}(R)$	`SELECT A, SUM(B) FROM R GROUP BY A`
Sorting	$\tau_{A\,\mathrm{DESC}}(R)$	`SELECT * FROM R ORDER BY A DESC`

Key Insight

Restriction (σ) selects *rows*; projection (π) selects *columns*. In SQL both ideas are combined into a single `SELECT ...FROM ...WHERE` statement.

Relational Algebra to SQL – Binary Operators

Operation	RA Syntax	SQL Equivalent
Union	$R_1 \cup R_2$	`SELECT ... UNION SELECT ...`
Intersection	$R_1 \cap R_2$	`SELECT ... INTERSECT SELECT ...`
Difference	$R_1 \setminus R_2$	`SELECT ... EXCEPT SELECT ...`
Cartesian Product	$R_1 \times R_2$	`SELECT * FROM R1, R2`
Division	$R_1 \div R_2$	No direct equivalent (use `NOT EXISTS`)

Notes

- `UNION` removes duplicates by default; use `UNION ALL` to keep them (faster, but includes duplicates).
- `INTERSECT` and `EXCEPT` are not available in all DBMSs (MySQL added `INTERSECT` / `EXCEPT` only in version 8.0.31).
- The Cartesian product (`FROM R1, R2` without a `WHERE` join condition) returns every combination of rows – usually a mistake in production queries.

Relational Algebra to SQL – Join Operators

Operation	RA	SQL
Natural Join	$R_1 \bowtie R_2$	`R1 NATURAL JOIN R2`
Inner Join	$R_1 \bowtie_C R_2$	`R1 INNER JOIN R2 ON C`
Left Outer Join	$R_1 \bowtie_L R_2$	`R1 LEFT JOIN R2 ON C`
Right Outer Join	$R_1 \bowtie_R R_2$	`R1 RIGHT JOIN R2 ON C`
Full Outer Join	$R_1 \bowtie_{LR} R_2$	`LEFT JOIN UNION RIGHT JOIN`

Choosing the Right Join

- Use `INNER JOIN` when you only want rows with a match on **both** sides.
- Use `LEFT JOIN` when you want all rows from the left table, with `NULL` for unmatched right-side columns.
- MySQL does not support `FULL OUTER JOIN` directly; emulate it with `LEFT JOIN UNION RIGHT JOIN`.

SELECT Execution Order

SQL clauses are **written** in one order but **executed** in a completely different order. Internalising this is the key to writing correct, efficient queries.

Execution Steps (in order)

1. **FROM** (and JOINs) – identify and combine source tables
2. **WHERE** – filter individual rows
3. **GROUP BY** – collapse rows into groups
4. **HAVING** – filter groups
5. **SELECT** – compute output expressions and assign aliases
6. **DISTINCT** – remove duplicate rows
7. **UNION / INTERSECT / EXCEPT** – combine result sets
8. **ORDER BY** – sort the final result
9. **LIMIT / OFFSET** – restrict the output size

Why Does This Matter?

- Column aliases (assigned at step 5) **cannot** be used in `WHERE` (step 2) or `HAVING` (step 4).

Joins – Overview

SQL **JOINs** combine rows from two or more tables based on a related column. They are the SQL realisation of the relational-algebra join operators and are essential whenever your data is spread across multiple tables – which is almost always the case in a properly normalised schema.

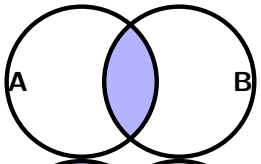
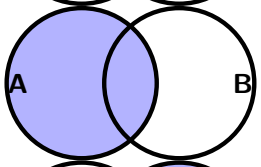
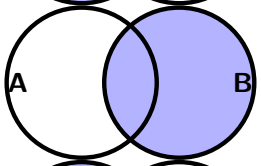
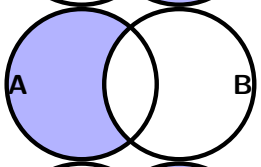
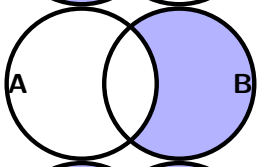
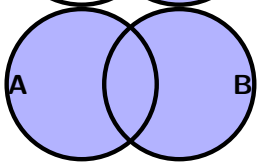
Without JOINs you would have to load every table separately and combine the data in application code, which is slow, error-prone, and defeats the entire purpose of a relational DBMS.

Join types covered in this section

- **INNER JOIN** – only matching rows from both sides
- **NATURAL JOIN** – automatic column-name matching
- **LEFT (OUTER) JOIN** – all rows from the left table
- **RIGHT (OUTER) JOIN** – all rows from the right table
- **Left Anti-Join** (semi-join via NULL trick)
- **Right Anti-Join** (semi-join via NULL trick)
- **FULL OUTER JOIN** – all rows from both tables

Think of a Venn diagram with two overlapping circles (A on the left, B on the

Join Types – Visual Overview

Join Type	Venn Diagram	Returns
INNER JOIN		Matching rows only
LEFT JOIN		All A + matched B
RIGHT JOIN		All B + matched A
Left Anti-Join		A rows with no match in B
Right Anti-Join		B rows with no match in A
FULL OUTER JOIN		All rows from both tables

How to read the diagrams

INNER JOIN

```sql
SELECT *
FROM   A
INNER JOIN B ON A.id = B.id;
```

What it does

Returns **only the rows where the join condition is satisfied in BOTH tables**. Any row in A that has no matching row in B is silently discarded – and vice versa.

- **Venn diagram**: only the overlapping centre of the two circles is returned.
- **Default join**: writing `JOIN` without a qualifier is equivalent to `INNER JOIN` in all major DBMSs (MySQL, PostgreSQL, SQL Server, Oracle).
- **Most commonly used join** in everyday SQL development.
- If the join column contains `NULL` values, those rows are *never* matched – `NULL = NULL` evaluates to `UNKNOWN`, not `TRUE`.

Examples

Use case Retrieve orders together with customer details: `orders INNER JOIN customers ON orders.customer_id = customers.id`
Only orders that have a valid, existing customer are returned.

NATURAL JOIN

```
SELECT *
FROM   A
NATURAL JOIN B;
```

What it does

Automatically joins on **all columns that share the same name** in both tables. No `ON` condition is written – the DBMS infers it. If both tables have a column `id`, the implicit condition is `A.id = B.id`.

- Duplicate joined columns appear **only once** in the result set (unlike INNER JOIN which would produce two `id` columns).
- Semantically equivalent to an INNER JOIN with an explicit `ON` clause naming every shared column.

Danger – implicit behaviour

If a developer later adds a new column to one of the tables and it happens to share a name with a column in the other table, the join condition changes *silently*. This is a hard-to-find bug. For production code, always prefer an explicit `INNER JOIN ... ON`.

LEFT (OUTER) JOIN

```sql
SELECT *
FROM   A
LEFT JOIN B ON A.id = B.id;
```

What it does

Returns **all rows from the LEFT table (A)**, plus the matching rows from B. Where a row in A has no match in B, all columns originating from B are filled with `NULL`.

- **Venn diagram**: the entire left circle plus the intersection.
- The keyword `OUTER` is optional – `LEFT JOIN` and `LEFT OUTER JOIN` are identical.
- The result always contains *at least* as many rows as table A.
- **Very commonly used**: whenever you want a complete list from one table and optionally enriched with data from another.

Examples

Use case `customers LEFT JOIN orders ON customers.id = orders.customer_id`

RIGHT (OUTER) JOIN

```sql
SELECT *
FROM   A
RIGHT JOIN B ON A.id = B.id;
```

What it does

Returns **all rows from the RIGHT table (B)**, plus the matching rows from A. Where a row in B has no match in A, all columns from A are filled with `NULL`.

- **Venn diagram**: the entire right circle plus the intersection.
- The keyword `OUTER` is optional.
- Every `RIGHT JOIN` can be rewritten as a `LEFT JOIN` by simply swapping the table order: `A RIGHT JOIN B` $\equiv$ `B LEFT JOIN A`.
- Because of this, `RIGHT JOIN` is relatively rare – most developers stick to `LEFT JOIN` for consistency and readability.

Examples

Use case `orders RIGHT JOIN products ON orders.product_id = products.id`
Returns *all* products, even those that have never been ordered. Orders that have

Left Anti-Join (NULL Trick)

```sql
-- Rows in A that have NO match in B
SELECT *
FROM   A
LEFT JOIN B ON A.id = B.id
WHERE  B.id IS NULL;
```

What it does

Returns only the rows from A that have **no matching row in B** whatsoever.

- The `LEFT JOIN` first produces all A rows, with `NULL` in B columns where no match exists.
- The `WHERE B.id IS NULL` filter then *keeps only those unmatched rows*.
- SQL has no dedicated `ANTI JOIN` or `SEMI JOIN` keyword – this two-step pattern is the standard workaround.
- **Venn diagram**: only the left crescent of circle A, excluding the intersection.
- Alternative using `NOT EXISTS` or `NOT IN` is possible but the NULL-trick LEFT JOIN often performs better.

Examples

Right Anti-Join (NULL Trick)

```sql
-- Rows in B that have NO match in A
SELECT *
FROM   A
RIGHT JOIN B ON A.id = B.id
WHERE  A.id IS NULL;
```

What it does

Returns only the rows from B that have **no matching row in A** whatsoever.

- Mirror image of the left anti-join: the `RIGHT JOIN` preserves all B rows, and the `WHERE A.id IS NULL` filter isolates those without a counterpart in A.
- **Venn diagram**: only the right crescent of circle B, excluding the intersection.
- Can equally be expressed as a `LEFT JOIN` with the tables swapped: `B LEFT JOIN A ON B.id = A.id WHERE A.id IS NULL`

Examples

Use case Find all departments that currently have no employees assigned: `employees RIGHT JOIN departments ON employees.dept_id = departments.id WHERE employees.id IS NULL`

FULL OUTER JOIN

```sql
-- MySQL does not support FULL OUTER JOIN directly.
-- Simulate it with a UNION of LEFT and RIGHT joins:
SELECT * FROM A LEFT  JOIN B ON A.id = B.id
UNION
SELECT * FROM A RIGHT JOIN B ON A.id = B.id
WHERE A.id IS NULL;
```

What it does

Returns **all rows from both tables**. Rows that have no match on the other side receive `NULL` in the columns originating from that side.

- **Venn diagram**: both circles entirely – the left crescent, the intersection, and the right crescent.
- PostgreSQL and SQL Server support `FULL OUTER JOIN` as a direct keyword. MySQL does not – the `UNION` trick above is required.
- The second `SELECT` uses `WHERE A.id IS NULL` to add *only* the right-side-only rows, avoiding duplication of the matched centre rows.

Examples

NATURAL JOIN and INNER JOIN

Joining Tables

```
-- Natural Join: automatically joins on all columns with the
     same name
SELECT * FROM addresses NATURAL JOIN cities;
-- implicitly: ON addresses.zip_code = cities.zip_code

-- Inner Join: explicitly specify the join condition
SELECT *
FROM employees AS e
INNER JOIN departments AS d ON e.department_id = d.id;
```

Comparison

- `NATURAL JOIN` is **convenient but fragile**: if someone later adds a column to either table whose name happens to match a column in the other table, the join silently changes its behaviour.
- `INNER JOIN ...ON` makes the join condition **explicit and stable** – always preferred in production code.

LEFT and RIGHT JOIN

Outer Joins – Keeping Unmatched Rows

```sql
-- Left Join: all rows from employees, matching rows from
    departments
SELECT e.name, d.name AS dept_name
FROM employees AS e
LEFT JOIN departments AS d ON e.department_id = d.id;
-- Employees without a department get NULL for dept_name

-- Right Join: all rows from departments, matching from
    employees
SELECT e.name, d.name AS dept_name
FROM employees AS e
RIGHT JOIN departments AS d ON e.department_id = d.id;
-- Departments with no employees still appear (NULL for e.
    name)
```

- `LEFT JOIN` is by far the most commonly used outer join – "give me everything from the left table, fill in NULLs where there is no match on the right".

INSERT INTO

Adding Rows to a Table

```sql
-- Insert a single row (explicit column list)
INSERT INTO unit (id, name) VALUES ('T', 'ton');

-- Insert multiple rows at once
INSERT INTO unit (id, name)
VALUES ('KG', 'kilogram'),
       ('G',  'gram'),
       ('L',  'litre');

-- Insert from a SELECT result
INSERT INTO archive_orders
SELECT * FROM orders WHERE order_date < '2020-01-01';
```

- Columns not listed in the column list receive their `DEFAULT` value, or `NULL` if no default is defined.
- `AUTO_INCREMENT` columns (typically the primary key) are **omitted** from the column list – the database assigns the next integer value automatically.
- When inserting into a table with **foreign key constraints**, the referenced row

UPDATE

Modifying Existing Rows

```sql
-- Update one column for a specific row
UPDATE employee
SET last_name = 'Miller'
WHERE id = 12;

-- Update multiple columns using expressions
UPDATE product
SET price         = price * 1.1,
    last_modified = NOW()
WHERE brand_id = 5;

-- DANGER: UPDATE without WHERE affects ALL rows!
UPDATE product SET price = 0; -- sets every product's price
    to zero!
```

Always Include a WHERE Clause

An `UPDATE` without `WHERE` modifies **every single row** in the table. Before running

DELETE

Removing Rows from a Table

```sql
-- Delete specific rows
DELETE FROM employee WHERE department_id = 5;

-- Delete with JOIN condition (MySQL syntax)
DELETE product
FROM product
INNER JOIN brand ON product.brand_id = brand.id
WHERE brand.country = 'USA';

-- DANGER: DELETE without WHERE empties the table!
DELETE FROM employee; -- removes every row!
```

- `DELETE` removes rows but **keeps the table structure** (schema, indexes, constraints). To remove the table entirely, use `DROP TABLE`.
- `TRUNCATE TABLE employee` is faster than `DELETE FROM employee` for emptying a table (it deallocates data pages directly), but it cannot be rolled back in some DBMSs and resets `AUTO_INCREMENT` counters.

CREATE DATABASE and CREATE TABLE

Defining the Schema

```sql
-- Create a new database and switch to it
CREATE DATABASE supermarket;
USE supermarket;

-- Create a table with various constraints
CREATE TABLE employee (
    id          INT AUTO_INCREMENT,
    username    VARCHAR(50)  NOT NULL UNIQUE,
    email       VARCHAR(255),
    age         INT,
    PRIMARY KEY (id),
    CHECK (age >= 18)
);
```

- `AUTO_INCREMENT`: MySQL automatically assigns the next integer value when a row is inserted without specifying this column.
- `NOT NULL`: the column cannot be omitted or explicitly set to `NULL` – the insert is rejected if it is.

Data Types

Choosing the correct data type saves storage, improves performance, and prevents data integrity errors.

Type	Description
`CHAR(n)`	Fixed-length string of exactly n characters
`VARCHAR(n)`	Variable-length string, max n characters
`BOOLEAN`	0 = false, any other value = true
`INT`	Integer (-2^{31} to $2^{31}-1$)
`DECIMAL(p,s)`	Exact decimal: p total digits, s after decimal
`FLOAT / DOUBLE`	Approximate floating-point (avoid for money!)
`DATE`	Date: `YYYY-MM-DD`
`TIME`	Time: `HH:MM:SS`
`TIMESTAMP`	Date+time: `YYYY-MM-DD HH:MM:SS`
`TEXT`	Long text (no index by default)
`BLOB`	Binary data (images, files)

Use DECIMAL for Money – Never FLOAT

`FLOAT` and `DOUBLE` store approximate values using binary floating-point. This can introduce rounding errors: 0.1 + 0.2 may not equal exactly 0.3. For monetary

ALTER TABLE and DROP

Modifying an Existing Table

```sql
-- Add a column
ALTER TABLE employee ADD email VARCHAR(255);

-- Rename a column
ALTER TABLE employee RENAME COLUMN email TO work_email;

-- Change the data type of a column
ALTER TABLE employee MODIFY COLUMN age TINYINT;

-- Remove a column
ALTER TABLE employee DROP COLUMN age;

-- Remove an entire table (irreversible!)
DROP TABLE employee;

-- Remove the entire database with all its tables!
DROP DATABASE supermarket;
```

Constraints

Defining Integrity Rules at Column and Table Level

```
CREATE TABLE product (
    id          INT AUTO_INCREMENT,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(255) UNIQUE,
    price       DECIMAL(10,2) DEFAULT 0.00,
    brand_id INT,
    PRIMARY KEY (id),
    FOREIGN KEY (brand_id) REFERENCES brand(id)
        ON DELETE SET NULL
        ON UPDATE CASCADE,
    CHECK (price >= 0)
);
```

Referential Actions

- `ON DELETE CASCADE` – if the referenced brand is deleted, **all its products are also deleted automatically**.
- `ON DELETE SET NULL` – if the brand is deleted, `brand_id` becomes `NULL`.

Named Constraints and ALTER TABLE ADD CONSTRAINT

Naming Constraints for Easier Management

```sql
CREATE TABLE employee (
    id     INT,
    email  VARCHAR(255),
    age    INT,
    CONSTRAINT pk_employee PRIMARY KEY (id),
    CONSTRAINT uq_email    UNIQUE (email),
    CONSTRAINT chk_age     CHECK (age >= 18)
);

-- Add a constraint to an existing table
ALTER TABLE employee
ADD CONSTRAINT fk_dept
FOREIGN KEY (dept_id) REFERENCES department(id);

-- Remove a named constraint
ALTER TABLE employee DROP CONSTRAINT chk_age;
```

VIEW, TRIGGER, PROCEDURE (!)

Advanced Database Objects

```sql
-- VIEW: a virtual table based on a SELECT
CREATE VIEW expensive_products AS
SELECT * FROM product WHERE price > 10;
DROP VIEW expensive_products;

-- TRIGGER: automatically execute SQL on a data change
CREATE TRIGGER set_default_bbd
BEFORE INSERT ON stock
FOR EACH ROW
SET NEW.best_before_date = DATE_ADD(NOW(), INTERVAL 30 DAY);

-- STORED PROCEDURE: a reusable, named SQL block
DELIMITER //
CREATE PROCEDURE raise_prices(IN pct DECIMAL(5,2))
BEGIN
    UPDATE product SET price = price * (1 + pct/100);
END//
DELIMITER ;
CALL raise_prices(10.0);
```

GRANT and REVOKE (DCL)

Controlling Access to Database Objects

```sql
-- Grant all privileges on a database to a user
GRANT ALL ON supermarket.* TO 'admin'@'localhost';

-- Grant only SELECT on the whole database
GRANT SELECT ON supermarket.* TO 'reader'@'%';

-- Grant specific privileges on a single table
GRANT INSERT, UPDATE ON supermarket.product
    TO 'editor'@'localhost';

-- Revoke a privilege
REVOKE INSERT ON supermarket.product
    FROM 'editor'@'localhost';
```

Principle of Least Privilege

- Grant only the **minimum permissions** a user or application actually needs to do its job.

Table of Contents

Why Backup and Recovery?

Data is one of the most valuable assets of any organisation. Hardware can be replaced in hours; data that is lost without a backup may be lost forever. Even a brief period of data unavailability can cause significant financial and reputational damage.

Every production database system **must** have a well-defined backup and recovery strategy before going live – not after the first incident.

Two key metrics for every recovery strategy

- **RTO – Recovery Time Objective**: the maximum acceptable duration of system downtime after a failure. "How long can we afford to be offline?"
- **RPO – Recovery Point Objective**: the maximum acceptable amount of data loss measured in time. "How much data can we afford to lose?"

Example: an online bank might have an RTO of 1 hour and an RPO of 0 minutes (no data loss permitted). A small internal tool might tolerate an RTO of 1 day and an RPO of 24 hours. Your backup strategy must be designed to meet these targets.

Backups

Definition

A **backup** is a copy of database data saved at a specific point in time. It is used to restore the database after data loss, corruption, accidental deletion, or hardware failure.

Common backup strategies – from coarsest to finest granularity:

- **Full backup (e.g. weekly)**: a complete copy of the entire database. Reliable and self-contained – restore from this alone is possible. Downside: slow to create and requires the most storage.
- **Differential backup (e.g. daily)**: copies only the data that has changed since the *last full backup*. Faster and smaller than a full backup; restore requires the last full backup plus the most recent differential.
- **Incremental backup (e.g. hourly)**: copies only data changed since the *last backup of any type*. Smallest and fastest; restore requires the full backup, every differential, and every incremental in sequence.
- **Transaction log backup**: saves a continuous log of every individual change. Enables **point-in-time recovery** – restore to any moment, not just the last backup time.

Recovery

Definition

Recovery is the process of restoring a database to a correct, consistent, and usable state after any type of failure.

Common failure scenarios:

- **Software issues**: application bugs that corrupt data, a transaction that completes partially before crashing, or a developer running `DELETE FROM orders` without a `WHERE` clause.
- **Hardware issues**: sudden power outage, disk head crash, server room fire or flood, failed storage controller.
- **Human error**: accidental `DROP TABLE`, updating the wrong rows, importing bad data that overwrites good data.

Standard recovery process (3 steps):

1. Restore the most recent **full backup** to get a consistent baseline.
2. Apply the most recent **differential backup** to bring data closer to the failure point.
3. Replay all **transaction log backups** in chronological order up to the desired recovery point in time.

Transaction Logs: Before- and After-Images

To support both rollback and crash recovery, every database engine maintains detailed change logs for every transaction.

Before-Image

The state of a data record **before** the modification. Created at the start of a transaction (before any change is applied). Used for `ROLLBACK` – if the transaction must be undone, the before-image is used to restore the original data. Kept in the log until the transaction either commits or rolls back.

After-Image

The state of a data record **after** the modification. Created when the change is applied to the in-memory cache. Used for **REDO** / crash recovery – if the system crashes after a commit but before the data was flushed to disk, the after-image allows the change to be replayed. Kept until the data has been physically written to disk (at a *checkpoint*).

Important: data modifications are first written to a **database cache** (RAM buffer) for performance. They are written to physical disk asynchronously during a *checkpoint*. If the system crashes between the commit and the next checkpoint

Transaction Lifecycle with Before- and After-Images

Concrete example: a transaction updates the price of "Cat Milk" from €1.99 to €2.49.

1. **START TRANSACTION**: the DBMS begins tracking this transaction.
2. **Create Before-Image**: the current record (price = 1.99) is written to the transaction log as the before-image.
3. **Modify data in cache**: the price is updated to 2.49 in the in-memory buffer. The physical disk file is *not yet changed*.
4. **Create After-Image**: the updated record (price = 2.49) is written to the transaction log as the after-image.
5. **COMMIT or ROLLBACK**:
   - **COMMIT**: the change is confirmed. The before-image is deleted from the log. The after-image remains until the buffer is flushed to disk at the next checkpoint, then it too is deleted.
   - **ROLLBACK**: the before-image is used to restore the record to price = 1.99. Both before- and after-image are discarded.

Why this matters

This mechanism directly implements two ACID properties: **Atomicity** (all-or-nothing via rollback) and **Durability** (committed changes survive crashes

RAID – Redundant Array of Independent Disks (!)

Definition (!)

RAID combines multiple physical hard drives into a single logical storage unit, providing a combination of improved performance (through parallelism) and/or fault tolerance (through redundancy), depending on the RAID level chosen.

- Can be implemented in **hardware** (dedicated RAID controller card with its own CPU and cache) or in **software** (the operating system manages RAID in firmware or kernel drivers). Hardware RAID is faster and more reliable.
- **Three fundamental techniques** used in different combinations by each RAID level:
  - **Striping**: data is split into blocks and written across multiple disks in parallel. Increases throughput – both reading and writing happen simultaneously on several disks.
  - **Mirroring**: an exact, real-time copy of all data is maintained on a second (or more) disk. If one disk fails, the mirror takes over immediately.
  - **Parity**: a mathematical checksum of the data is stored alongside it. If one disk fails, its content can be *reconstructed* from the remaining data and the parity information – without needing a full mirror copy.
- There is always a **trade-off** between performance, redundancy, and storage efficiency. No RAID level maximises all three simultaneously.

RAID 0 – Striping (!)

How it works: data is split into equal-sized blocks (*stripes*) and written across all disks in round-robin order.

Example with 2 disks: Disk 0 receives blocks A1, A2, A3, A4; Disk 1 receives A5, A6, A7, A8. Both disks are written simultaneously.

Pros:

- **Fastest read and write performance** of all RAID levels (full parallelism).
- **100% storage efficiency** – no space wasted on redundancy.

Cons:

- **Zero fault tolerance**: if even *one* disk fails, all data is lost (stripes from the failed disk are missing and cannot be reconstructed).
- RAID 0 is technically *not* redundant despite being called RAID.

Disk 0	Disk 1
A1	A5
A2	A6
A3	A7
A4	A8

RAID 1 – Mirroring (!)

How it works: every write operation is duplicated to two (or more) disks simultaneously. Disk 0 and Disk 1 are always byte-for-byte identical.

Example with 2 disks: Disk 0 = A1, A2, A3, A4 and Disk 1 = A1, A2, A3, A4.

Pros:

- **Full redundancy**: if one disk fails, the mirror takes over instantly with no data loss and no rebuild time.
- **Fast reads**: the controller can serve read requests from either disk in parallel, doubling read throughput.

Cons:

- **50% storage efficiency**: you pay for two disks but only get the capacity of one.
- Most expensive per usable gigabyte of all RAID levels.

Disk 0	Disk 1
A1	A1
A2	A2
A3	A3
A4	A4

RAID 5 – Striping with Distributed Parity (!)

How it works: data is striped across $n \geq 3$ disks. For each stripe, a *parity block* is computed and stored on one of the disks (the parity location rotates across disks so no single disk becomes the bottleneck).

If one disk fails, its data can be **mathematically reconstructed** from the remaining disks and the parity blocks using XOR operations.

Pros:

- Good balance of **performance**, **storage efficiency**, and **fault tolerance**.
- Storage efficiency: $\frac{n-1}{n}$ – with 4 disks, 75% of total capacity is usable.
- Survives the failure of **exactly one disk** without data loss.

Cons:

- **Slower writes** than RAID 0 or RAID 1 (parity must be computed for every write).
- If **two disks fail simultaneously**, all data is lost (no second parity).
- After a disk failure, **rebuild time** can take hours or days on large arrays – during which the remaining disks are under extra stress.

Use case (!)

The most common RAID level in enterprise NAS and SAN storage arrays where a balance of cost, performance, and safety is needed.

Table of Contents

What is Concurrency?

Definition

Concurrency in the context of databases refers to the ability of multiple users or processes to access and modify the same database data **simultaneously**, without compromising data integrity or consistency.

In any real production system – an online shop, a banking application, an ERP – hundreds or thousands of transactions may be executing at the exact same moment. This is not an edge case; it is the normal operating mode.

Without proper concurrency control, the following problems arise:

- Users overwrite each other's changes (*lost update*).
- A user reads data that is about to be rolled back (*dirty read*).
- A user reads the same data twice and gets different results (*non-repeatable read*).
- A query sees a different set of rows on two successive executions (*phantom read*).

Concurrency control is one of the core responsibilities of any DBMS. The goal is to allow maximum parallelism while guaranteeing that the result is always equivalent to some *serial* (one-at-a-time) execution of those same transactions – a

Problem 1: Lost Update

Scenario: two users simultaneously update the unit price of the same product.

User A	**User B**	**What happens?**
Read price (= 10.00)		
	Read price (= 10.00)	Both see old value
Compute 10.00 + 2.00		
	Compute 10.00 − 1.00	
Write price = 12.00		A commits first
COMMIT		
	Write price = 9.00	B overwrites A!
	COMMIT	A's change is LOST

Result

Both users read the same original value (10.00), calculated their own updates independently, and the *last writer wins*. User A's increase to 12.00 is completely overwritten and lost. The final price is 9.00 instead of the logically correct 11.00 (or 8.00, depending on which update was intended to be the "final" one).

Problem 2: Dirty Read (Uncommitted Dependency)

Scenario: an ATM reads an account balance that is subsequently rolled back.

ATM	Bank Server	What happens?
	BEGIN: incoming transfer of €500 Balance: 300 + 500 = 800 (in cache)	
Read balance = 800		Reads *dirty* data
COMMIT (displays €800)		
	ROLLBACK (transfer failed)	Balance reverts to 300
The ATM showed €800 but the real balance is €300.		

Result

The ATM read data from a transaction that had not yet committed – "dirty" (uncommitted) data. When the bank server rolls back, the ATM has already confirmed a balance that no longer exists. This can lead the customer to withdraw money they do not actually have.

Problem 3: Inconsistent Analysis (Non-Repeatable / Phantom Read)

Scenario: an auditor totals three account balances while a transfer runs concurrently.
Initial state: Account 1 = €400, Account 2 = €300, Account 3 = €700 (total = €1 400).

User A (audit)	**User B (transfer)**	**Event**
Read Acc 1 = 400		
Read Acc 2 = 300		
	Acc 3: 700 − 600 = 100	Transfer begins
	Acc 1: 400 + 600 = 1 000	
	COMMIT	
Read Acc 3 = 100		Sees new value!
Sum = 400+300+100 = **800**		**Wrong! Should be 1 400**

Result

User A read the three accounts at *different points in time* and saw a snapshot that

Concurrency Control Strategies

Two fundamentally different philosophies for dealing with concurrent access:

Optimistic Concurrency Control (OCC)

Assumption: conflicts are rare. Let all transactions proceed freely.

- Transactions read and write without acquiring any locks.
- At `COMMIT` time, the DBMS checks whether a conflict occurred.
- If a conflict is detected, the transaction is **aborted and retried**.
- Pros: simple to implement; high throughput when conflicts are actually rare.
- Cons: wasted work on conflict; poor performance under high contention.

Pessimistic Concurrency Control (Lock-Based)

Assumption: conflicts are likely. Prevent them before they happen.

- A transaction must **acquire a lock** on a data item before accessing it.
- Other transactions that need the same item must **wait** for the lock to be released.
- Locks are released at transaction end (commit or rollback).

Lock Granularity

A **lock** can be placed at different levels of the data hierarchy. The *granularity* of a lock determines how much data is protected – and how much concurrency is sacrificed.

Level	Description	Typical Use Case
Database	Locks the entire database; only one user can access it at a time	System maintenance, bulk import, schema migrations
Table	Locks a single table; all rows are blocked for other writers	Few-user systems; batch operations on one table
Row (Tuple)	Locks a single row; other rows in the same table are freely accessible	Standard production OLTP systems (MySQL InnoDB, PostgreSQL)
Cell (Field)	Locks a single attribute value within a row	Extremely rare; very complex to implement

Design principle

Always lock **the smallest amount of data necessary** to maintain correctness. Coarser locks are easier to implement but destroy concurrency. Fine-grained row-level locks maximise parallelism but require more overhead to track.

Shared and Exclusive Locks

Exclusive Lock (X-Lock) – Write Lock

Acquired by `UPDATE`, `INSERT`, and `DELETE` operations. While an X-lock is held on a data item, **no other transaction may acquire any lock** (neither shared nor exclusive) on that item. Writes require exclusive access.

Shared Lock (S-Lock) – Read Lock

Acquired by `SELECT` operations. Multiple transactions may hold S-locks on the same item simultaneously – concurrent reads are always safe. However, **an X-lock cannot be granted** while any S-lock is held, because a write while others are reading would produce inconsistent results.

Lock compatibility matrix:

Requested \ Held	**S-Lock held**	**X-Lock held**
Request S-Lock	Granted	Wait
Request X-Lock	Wait	Wait

Examples

Deadlocks

Definition

A **deadlock** occurs when two or more transactions are each waiting for the other to release a lock – creating a circular dependency from which no transaction can escape on its own.

Classic two-transaction deadlock:

- Transaction A holds an X-lock on Row 1, now requests a lock on Row 2.
- Transaction B holds an X-lock on Row 2, now requests a lock on Row 1.
- A is waiting for B; B is waiting for A. Neither can proceed. Ever.

Detection and resolution – two main approaches:

- **Deadlock detection**: the DBMS maintains a "waits-for" graph (nodes are transactions, edges are "is waiting for"). Periodically check for *cycles* in this graph. When a cycle is found, one transaction (the *victim*) is aborted and retried; the others can then proceed. MySQL InnoDB and PostgreSQL use this approach.
- **Deadlock prevention**: enforce a global ordering on resources and require all transactions to acquire locks in that fixed order. If transaction A always locks Row 1 before Row 2, and B does the same, no circular wait can form.

Table of Contents

Why Query Optimization Matters

A correct query that runs in 30 seconds is useless in a web application that must respond in under 200 ms. As data volumes grow, the difference between an optimized and an unoptimized query can be the difference between seconds and hours.

Two Levels of Optimization

1. **Query-level**: rewrite the SQL to express the same result more efficiently – reduce the number of rows processed as early as possible.
2. **Schema-level**: add or adjust **indexes** so the database engine can locate the required rows without scanning the entire table.

The golden rule

Measure before you optimize. Use `EXPLAIN` / `EXPLAIN ANALYZE` to find the real bottleneck – do not guess. Optimizing the wrong query wastes time and can even make things worse.

How a Query is Executed – The Query Planner

When you send a `SELECT` statement, the DBMS does not execute it literally. Instead, a **query planner (optimizer)** produces an *execution plan* – a tree of physical operations that produce the same result as efficiently as possible.

Typical plan steps

1. **Parsing**: validate syntax, resolve table/column names
2. **Logical rewriting**: push predicates down, eliminate redundancy, reorder joins
3. **Statistics lookup**: the planner uses *table statistics* (row count, column cardinality, histogram) to estimate costs
4. **Physical plan selection**: choose between full table scan, index scan, nested-loop join, hash join, etc.
5. **Execution**: run the chosen plan and stream results

Examples

Key insight The planner can only choose from the access paths that *exist*. Adding an index creates a new, cheaper access path for the planner to choose.

EXPLAIN and EXPLAIN ANALYZE

Inspecting the Execution Plan

```
-- Show the query plan (estimated costs, no actual execution
    )
EXPLAIN
SELECT * FROM product WHERE brand_id = 5;

-- Execute the query AND show actual timing + row counts
EXPLAIN ANALYZE
SELECT p.name, b.name
FROM   product p
JOIN   brand b ON p.brand_id = b.id
WHERE  p.price > 10;
```

What to look for in the output

- **Full Table Scan** (`ALL` in MySQL): the engine reads every row – a red flag for large tables.
- **Index Scan / Index Range Scan**: the engine navigates an index tree – much faster.

Indexes – What They Are

An **index** is a separate data structure maintained by the DBMS alongside a table. It maps column values to the physical location of the corresponding row, enabling fast lookups without scanning every row.

Analogy

Think of the index at the back of a textbook. Instead of reading every page to find "normalization", you look up the word in the index and jump directly to the right page. A database index works the same way.

Trade-offs

Benefit	**Cost**
Fast `SELECT` / `WHERE`	Extra disk space
Fast `JOIN` on foreign keys	Slower `INSERT`/`UPDATE`/`DELETE`
Fast `ORDER BY` / `GROUP BY`	Must be maintained by DBMS

Primary and Unique keys are automatically indexed

MySQL always creates a **clustered B-tree index** on the primary key. `UNIQUE`

B-Tree Index – How It Works

The default index type in MySQL (and most relational DBMSs) is a **B-Tree (Balanced Tree)** index.

Structure

- The tree is sorted by the indexed column value.
- The **root** splits the value range into subtrees.
- **Internal nodes** further narrow the range.
- **Leaf nodes** store the actual indexed values plus pointers to the table rows (row IDs / primary key values).
- The tree is always balanced – every leaf is at the same depth, so lookup time is $O(\log n)$ regardless of which value is searched.

What B-Tree indexes support

- **Equality**: `WHERE city = 'Berlin'` – point lookup
- **Range**: `WHERE price BETWEEN 5 AND 20` – range scan
- **Prefix**: `WHERE name LIKE 'Mül%'` – left-anchored
- **Sorting**: `ORDER BY` on indexed columns can skip filesort

Creating and Dropping Indexes

Syntax

```sql
-- Create a simple index on one column
CREATE INDEX idx_product_price ON product(price);

-- Create a composite (multi-column) index
CREATE INDEX idx_product_brand_price
    ON product(brand_id, price);

-- Create a unique index (also enforces uniqueness
    constraint)
CREATE UNIQUE INDEX uq_product_sku ON product(sku);

-- Show all indexes on a table
SHOW INDEX FROM product;

-- Remove an index
DROP INDEX idx_product_price ON product;
```

When the planner uses an index

Composite Indexes and Column Order

A **composite index** covers multiple columns. The order of columns in the index matters enormously.

The leftmost prefix rule

A composite index on `(a, b, c)` can be used for:

- `WHERE a = ?` – uses index
- `WHERE a = ? AND b = ?` – uses index
- `WHERE a = ? AND b = ? AND c = ?` – uses index
- `WHERE b = ?` – **does NOT use index** (b is not the leftmost column)
- `WHERE a = ? AND c = ?` – uses index on a only, then filters c manually

Examples

Design tip Put the most selective (highest cardinality) column first. If queries filter by `brand_id` then optionally by `price`, the index should be `(brand_id, price)`, not `(price, brand_id)`.

Covering Indexes

A **covering index** is an index that contains *all* the columns a query needs – so the DBMS can answer the query entirely from the index without touching the table rows at all.

Example

Query: `SELECT price FROM product WHERE brand_id = 5;`

If the index is `(brand_id, price)`, the engine finds the matching `brand_id` entries in the index and reads `price` directly from the index leaf nodes. **No row lookup required.**

When does it help most?

- The table is very wide (many columns) but the query only needs a few
- The table has many rows and the query is executed frequently
- The `SELECT` list exactly matches the indexed columns

EXPLAIN output shows `Extra: Using index` when a covering index is used.

When NOT to Use an Index

Indexes are not always the answer. Knowing when to *avoid* them is equally important.

Situations where indexes hurt more than they help

- **Very small tables**: a full scan of 500 rows is faster than an index lookup + row fetch.
- **Low-cardinality columns**: a `BOOLEAN` column or a `gender` column with only 2–3 distinct values. An index here hardly reduces the rows scanned.
- **Columns modified very frequently**: every `UPDATE`/`INSERT`/`DELETE` must update the index too. On write-heavy tables, too many indexes slow down writes significantly.
- **Functions on indexed columns**: `WHERE YEAR(created_at) = 2024` defeats the index on `created_at`. Rewrite as `WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'`.

Common Query Anti-Patterns

Patterns that prevent index use or waste resources

```sql
-- Anti-pattern 1: function on indexed column
WHERE UPPER(name) = 'MILLER'          -- index on name not used
-- Fix: store data in normalized case, or use a functional
    index

-- Anti-pattern 2: leading wildcard
WHERE name LIKE '%miller%'            -- full table scan
-- Fix: use FULLTEXT index for free-text search

-- Anti-pattern 3: implicit type conversion
WHERE id = '42'      -- id is INT, '42' is VARCHAR -> may skip
     index
-- Fix: match the data type: WHERE id = 42

-- Anti-pattern 4: SELECT * on large tables
SELECT * FROM orders;                 -- pulls every column,
    every row
-- Fix: always select only the columns you actually need
```

The N+1 Query Problem

The **N+1 problem** is one of the most common performance pitfalls in applications that use ORMs or generate SQL in loops.

What it is

```sql
-- 1 query to get all orders
SELECT * FROM orders;    -- returns N rows

-- Then N queries, one per order, to get the customer:
SELECT * FROM customer WHERE id = 1;
SELECT * FROM customer WHERE id = 2;
-- ... N more queries
```

The fix: one JOIN instead of N+1 queries

```sql
-- Single query returns everything needed
SELECT o.id, o.total, c.name
FROM   orders o
JOIN   customer c ON o.customer_id = c.id;
```

Query Optimization – Practical Checklist

Step-by-step approach

1. **Run `EXPLAIN ANALYZE`** – identify the slowest operation in the plan (high `rows`, `ALL` access, `Using filesort`).
2. **Check for missing indexes** on `WHERE`, `JOIN ON`, and `ORDER BY` columns.
3. **Avoid functions on indexed columns** in `WHERE` clauses – rewrite the predicate instead.
4. **Select only needed columns** – drop `SELECT *`; use covering indexes where possible.
5. **Push filters early** – filter rows as close to the source table as possible, before joins.
6. **Replace correlated subqueries** with `JOIN` or window functions.
7. **Batch writes**: wrap many small `INSERT` statements in a single multi-row `INSERT` or a transaction.
8. **Update statistics**: run `ANALYZE TABLE` periodically so the planner has fresh data.

Table of Contents

Database Security – Overview

Database security encompasses all policies, controls, and measures implemented to protect database contents, services, and infrastructure from unauthorised access, misuse, corruption, and accidental damage.

Key threat vectors every database administrator must address:

- **SQL Injection attacks** – malicious code injected through user input fields
- **Excessive user privileges** – accounts with far more permissions than needed
- **Weak authentication** – default passwords, no lockout policy, shared accounts
- **Unencrypted connections** – data sent in plaintext over the network
- **Unencrypted stored data** – sensitive columns (passwords, PII) stored in plain text
- **Missing audit logs** – no record of who accessed or modified what data, when
- **Unpatched software** – known CVEs in older DBMS versions exploited by attackers

Focus of this section

We will focus in depth on the most widespread and destructive attack vector: **SQL Injection** – and then introduce **Object-Relational Mapping (ORM)** as one of the most effective architectural defences against it.

SQL Injection – What Is It?

Definition

SQL Injection is a code injection attack in which an attacker inserts ("injects") malicious SQL code into a user-facing input field. If the application builds SQL queries by string concatenation, the injected code becomes part of the executed query, causing the DBMS to perform unintended operations.

- First publicly documented in **1998** by security researcher Jeff Forristal (writing under the alias "rfp") in the online magazine *Phrack*.
- Consistently ranked in the **OWASP Top 10** most critical web application security risks – it has appeared on this list every year since the list was created.
- Exploits a fundamentally simple mistake: **user-supplied data is treated as code** rather than as a literal value.
- Consequences range from unauthorised data disclosure (reading all users' records) to data modification or deletion, authentication bypass, and in extreme cases remote code execution on the database server.

Examples

Why is it still so common? SQL Injection is easy to make (one line of careless

SQL Injection – How It Works: Comment Attack

Vulnerable login query (built by string concatenation):

```
SELECT *
FROM   users
WHERE  username = '$username'
AND    password = '$password'
```

Attack 1 – Comment injection:

Attacker enters: username = `admin' --` password = `anything`

The application builds and executes the following query:

```
SELECT *
FROM   users
WHERE  username = 'admin' -- ' AND password = 'anything'
```

The double-dash (–) is a SQL comment delimiter. Everything after it is ignored by the DBMS. The `AND password = 'anything'` clause is effectively **commented out**.

Result

The query returns the admin user row *without checking the password at all*. The

SQL Injection – OR 1=1 Attack

Attack 2 – Always-true condition:

Attacker enters: username = `' OR 1=1 -` password = `anything`

The application builds and executes:

```
SELECT *
FROM   users
WHERE  username = '' OR 1=1 -- ' AND password = 'anything'
```

The injected `OR 1=1` clause makes the entire `WHERE` condition evaluate to `TRUE` for *every row* in the table, because `1=1` is always true and `OR` only requires one side to be true.

Result

The query returns **all rows in the users table**. The attacker is logged in as the first user returned – which is often the administrator account with the lowest ID or the first entry in the table.

Other variants

Attackers can also use `UNION SELECT` to extract data from other tables, use `DROP TABLE` to destroy data, or use `INSERT` to create backdoor accounts. The principle

Real-World SQL Injection Cases

These attacks caused billions of dollars in damages – and all were preventable

- **Heartland Payment Systems (2008)**: Over 100 million credit and debit card numbers were stolen via SQL injection in the company's internal payment processing network. Heartland paid over $140 million in settlements. At the time, it was the largest data breach in US history.
- **Sony Pictures (2011)**: More than 1 million user accounts (including plaintext passwords, email addresses, and home addresses) were compromised. The attacker used a single SQL injection vulnerability in a sweepstakes/contest page. Sony's reputation suffered enormously.
- **TalkTalk (2015)**: Personal and financial data of 157,000 UK customers was exposed via SQL injection on TalkTalk's website. The UK Information Commissioner's Office (ICO) fined TalkTalk £400,000. A 17-year-old was subsequently convicted for the attack.

In every case, **a single parameterised query** in place of string concatenation would have made the injection impossible. The fix is cheap; the attack is catastrophically expensive.

Preventing SQL Injection – Parameterised Queries

The most effective and fundamental defence: **never construct SQL queries by concatenating user input**. Always use **parameterised queries** (also called prepared statements).

```
-- VULNERABLE: direct string concatenation
"SELECT * FROM users WHERE name = '" + username + "'"

-- SAFE: parameterised query in Python (psycopg2 / MySQLdb)
cursor.execute(
    "SELECT * FROM users WHERE name = %s",
    (username,)
)

-- SAFE: Java PreparedStatement
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM users WHERE name = ?"
);
ps.setString(1, username);
ResultSet rs = ps.executeQuery();
```

Why parameterised queries are safe

Preventing SQL Injection – Configuration and Privileges

Parameterised queries are the primary defence. The following hardening measures provide additional layers of protection:

Database hardening checklist

- **Rename privileged accounts**: do not use default names like `admin`, `root`, or `sa` – attackers specifically target these names.
- **Principle of Least Privilege**: the database account used by the application should have *only* the permissions it genuinely needs (e.g. `SELECT` and `INSERT` on specific tables; never `DROP`, `CREATE`, or `GRANT`).
- **Disable unused default accounts**: DBMSs create several built-in user accounts during installation. Remove or disable any that the application does not need.
- **Server-side input validation**: validate and sanitise all input on the server (never trust client-side JavaScript validation – it can be bypassed trivially).
- **Suppress error messages**: never expose raw SQL error messages to end users; they reveal table names, column names, and database structure to attackers.
- **Encrypt sensitive data**: passwords *must* be stored as salted hashes (e.g.

Table of Contents

Object-Relational Mapping (ORM)

Definition

An **ORM (Object-Relational Mapper)** is a software library that automatically translates between the relational database world (tables, rows, columns) and the object-oriented programming world (classes, instances, fields), allowing developers to work with database data using the idioms of their programming language rather than writing raw SQL.

Conceptual mapping:

Relational Database Concept	**Object-Oriented Equivalent (Java)**
Relation (Table)	Class
Tuple (Row)	Instance (object) of the class
Attribute (Column)	Field / property of the class
Primary Key	`id` field (often auto-generated)
Foreign Key	Reference (object pointer) to another class instanc
Stored Procedure	Method on the class or repository

Popular ORM frameworks: **Hibernate** and **JPA** (Java), **SQLAlchemy** (Python), **ActiveRecord** (Ruby on Rails), **Eloquent** (PHP / Laravel), **Entity Framework** (.NET)

ORM – Why Use It?

- **Built-in SQL injection protection**: ORMs use parameterised queries internally – SQL is *never* constructed by string concatenation. Correctly used, an ORM makes SQL injection structurally impossible.
- **Developer productivity**: instead of writing repetitive `INSERT`, `UPDATE`, `SELECT` boilerplate, you call methods like `repository.save(employee)` or `repository.findById(42)`.
- **Database portability**: the ORM abstracts away DBMS-specific SQL dialects. Switching from MySQL to PostgreSQL often requires only a configuration change, not rewriting queries.
- **Schema-as-code**: the database schema is defined as Java/Python/C# classes. Changes can be tracked in version control alongside application code; migrations can be generated automatically.
- **Relationships handled automatically**: `@ManyToOne`, `@OneToMany` annotations define join behaviour; the ORM generates the correct JOIN queries transparently.

Trade-offs

ORMs can generate suboptimal or overly complex SQL for non-trivial queries. For performance-critical operations (bulk inserts, complex aggregations), dropping down to **native SQL** through the ORM's escape hatch is common and

Jakarta Persistence API (JPA)

JPA – formerly Java Persistence API

The **standard Java/Jakarta EE specification** for ORM. JPA defines the annotations and programming model; **Hibernate** is by far the most popular implementation. Using JPA keeps your code vendor-neutral – you could switch from Hibernate to EclipseLink with minimal changes.

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name", nullable = false)
    private String firstName;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
```

Spring Data JPA – Repository Pattern

Spring Data JPA builds on top of JPA/Hibernate and eliminates almost all boilerplate CRUD code. You define an interface; Spring generates the implementation at runtime.

```
// Extend JpaRepository<EntityType, PrimaryKeyType>
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {

    // Spring derives the SQL from the method name:
    List<Employee> findByLastName(String lastName);
    List<Employee> findByDepartmentNameAndAgeGreaterThan(
        String deptName, int age);

    // Use JPQL for complex queries
    @Query("SELECT e FROM Employee e WHERE e.salary > :min")
    List<Employee> findHighEarners(@Param("min") double
        minSalary);
}
```

What Spring provides for free

save(entity) findById(id) findAll() deleteById(id) count()

ORM – Lazy vs Eager Loading

When an `Employee` entity has a `@ManyToOne` link to `Department`, the ORM must decide *when* to load the related object.

Two loading strategies

- **EAGER loading**: the related entity is loaded *immediately* together with the parent, always via a `JOIN`. Simple but can load far more data than needed.
- **LAZY loading**: the related entity is loaded *on first access* (a second query is issued only when `employee.getDepartment()` is called). Default for `@OneToMany` and `@ManyToMany` in JPA.

```
@ManyToOne(fetch = FetchType.LAZY)    // default for *ToOne is EAGER
@JoinColumn(name = "department_id")
private Department department;
```

Lazy loading and the N+1 problem

LAZY loading is the root cause of the N+1 query problem in ORM frameworks. When iterating over a list of employees and accessing `getDepartment()` inside the loop, each call fires a separate SQL query. Fix: use a `JOIN FETCH` query or

Table of Contents

ACID – The Gold Standard for Transactions

ACID Properties

The four guarantees that define a **reliable database transaction** in classical relational database systems:

- **Atomicity**: a transaction is *all-or-nothing*. Either every single operation within the transaction succeeds and is applied, or none of them are. There is no intermediate, partially-applied state. Implemented via before-images and rollback.
- **Consistency**: a transaction always brings the database from one *valid state* to another. All integrity constraints (primary keys, foreign keys, check constraints, triggers) must hold before the transaction starts and after it ends. A transaction that would violate a constraint is aborted entirely.
- **Isolation**: concurrent transactions behave *as if they were executed one at a time* in some serial order. The intermediate, in-progress state of one transaction is not visible to any other transaction. Implemented via locks or multi-version concurrency control (MVCC).
- **Durability**: once a transaction successfully commits, its changes are *permanent* – even if the system crashes, loses power, or fails immediately afterward. Implemented via the write-ahead transaction log (after-images)

CAP Theorem

CAP Theorem (Eric Brewer, 2000)

Any **distributed** data store can simultaneously guarantee **at most two** of the following three properties:

- **Consistency (C)**: every read receives the *most recent write*, or an error. All nodes in the distributed system see the same data at the same time.
- **Availability (A)**: every request sent to a non-failing node receives a *response* – not necessarily the most recent data, but *some* response (no timeouts or errors due to the distributed nature).
- **Partition Tolerance (P)**: the system continues to *operate correctly* even if some network messages between nodes are dropped, delayed, or the network splits into isolated segments (a *network partition*).

The Catch – Partitions Always Happen

In any real distributed system deployed across multiple machines or data centres, network partitions are not hypothetical – they *will* happen at some point (network cables fail, switches reboot, data centres lose connectivity). Therefore, **Partition Tolerance is not optional** in practice. Every distributed database *must* tolerate partitions, meaning the real trade-off is always between **Consistency** and

CAP Theorem – Practical Implications

When a network partition occurs, the distributed system must make a choice:

- **Sacrifice Availability, maintain Consistency (CP)**: cancel or reject operations that cannot be guaranteed to be consistent across all nodes. The system returns an error rather than a potentially stale answer. Example: MongoDB (in default configuration) refuses writes to a partition-isolated node.
- **Sacrifice Consistency, maintain Availability (AP)**: continue serving requests from all nodes even if some return stale (out-of-date) data. The system will *eventually* converge once the partition heals. Example: Cassandra continues to serve reads and writes from all nodes; reconciliation happens later.

Classification of common database systems:

Category	**Examples**	**Trade-off**
CA (no partition tolerance)	MySQL, PostgreSQL, Oracle	Ideal for single-node; not truly distributed
CP (consistent, part.-tolerant)	MongoDB, HBase, Zookeeper	May reject requests during partition
AP (available, part.-	DynamoDB, CouchDB,	May return stale data

What is NoSQL?

Definition

NoSQL ("Not only SQL") refers to a broad class of database management systems designed for specific use cases that are not well-served by traditional relational databases. They typically sacrifice strict ACID compliance in exchange for horizontal scalability, schema flexibility, or specialised data models.

Why NoSQL emerged – limitations of relational DBs at scale:

- **Horizontal scalability**: relational databases are hard to shard across hundreds of commodity servers. NoSQL databases are designed from the ground up to distribute.
- **Schema flexibility**: in fast-moving web applications, data structures change frequently. Relational schemas require `ALTER TABLE` migrations; document stores accept any JSON structure.
- **High write throughput**: logging millions of IoT events or user actions per second would overwhelm a single relational DB; distributed NoSQL clusters handle this natively.
- **Specialised data models**: graphs, documents, and key-value pairs map very poorly to flat relational tables.

BASE – The NoSQL Alternative to ACID

BASE Properties

Most NoSQL databases follow the **BASE** model instead of ACID. BASE deliberately relaxes consistency guarantees in exchange for availability and performance:

- **Basically Available (BA)**: the system is *always available* to accept requests. All users can access data concurrently at any time, without waiting for locks or transactions to complete. The system prioritises responding to every request over guaranteeing the freshness of the response.
- **Soft State (S)**: the state of the system may change *over time even without new user input*, as replication and consistency updates propagate across nodes in the background. Data may be transient – different nodes may hold different versions of the same record temporarily.
- **Eventually Consistent (E)**: after all updates have stopped and sufficient propagation time has passed, all replicas *will converge* to the same value. The system does NOT guarantee that all nodes see identical data at the same instant – only that they will agree eventually.

When is BASE acceptable?

ACID vs. BASE – Comparison

Property	ACID	BASE
Primary focus	Data correctness and integrity	Availability and performance
Consistency	Guaranteed immediately after every commit	Eventually consistent (may lag)
Transactions	Full ACID transactions with rollback	Usually none, or limited / per-document
State during ops	Always consistent; invalid states impossible	Soft state; temporary inconsistency is expected
Scalability	Vertical (scale up: more CPU/RAM)	Horizontal (scale out: more servers)
Use cases	Finance, ERP, healthcare, banking	Social media, IoT, event logging, caches
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, Redis, DynamoDB

Practical takeaway

These are not competing philosophies where one is "better" – they are different